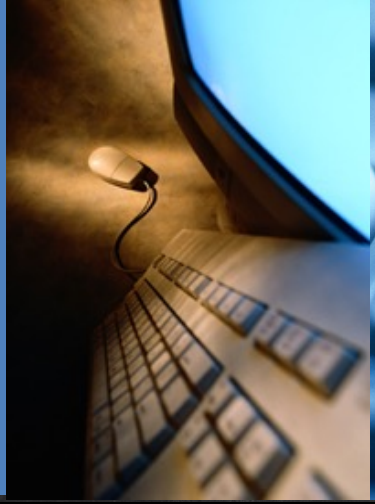
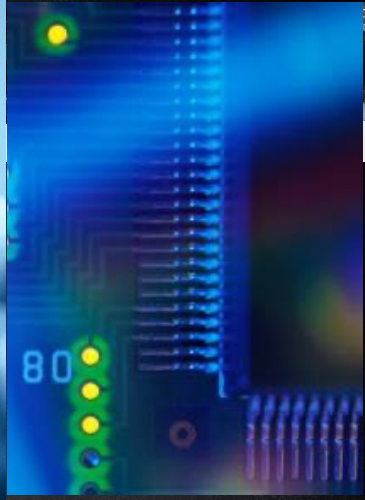


分布式系统
Distributed Systems

陈鹏飞
计算机学院
chenpf7@mail.sysu.edu.cn
办公室：学院楼310
主页：http://sdcs.sysu.edu.cn/node/3747







第十一讲 — 大数据+分布式机器学习



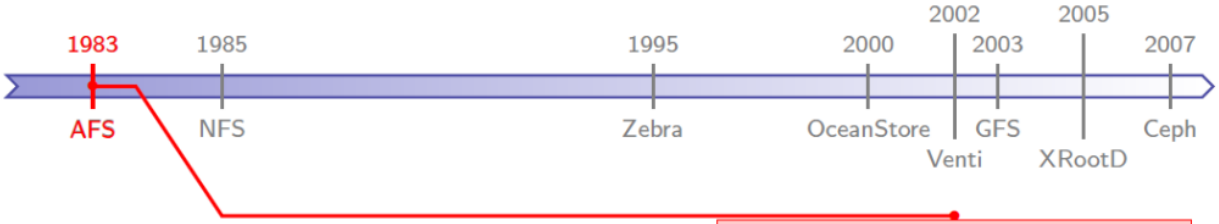




分布式文件系统

补充内容

Milestones in Distributed File Systems



"AFS was the first safe and efficient distributed computing system, available [...] on campus. It was a clear precursor to the Dropbox-like software packages today. [...] [It] allowed students (like Drew Houston and Arash Ferdowsi) access to all their stuff from any connected computer."

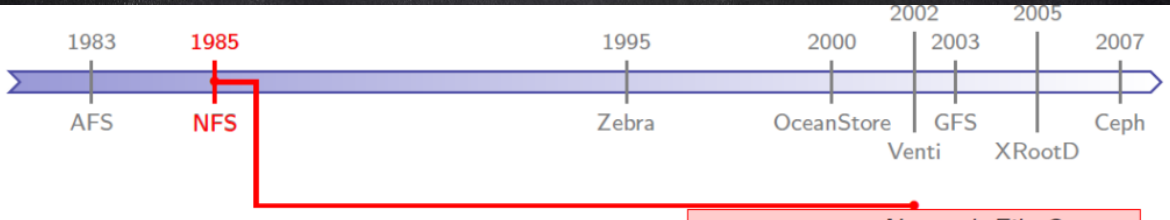
http://www.wired.com/2011/12/backdrop-dropbox

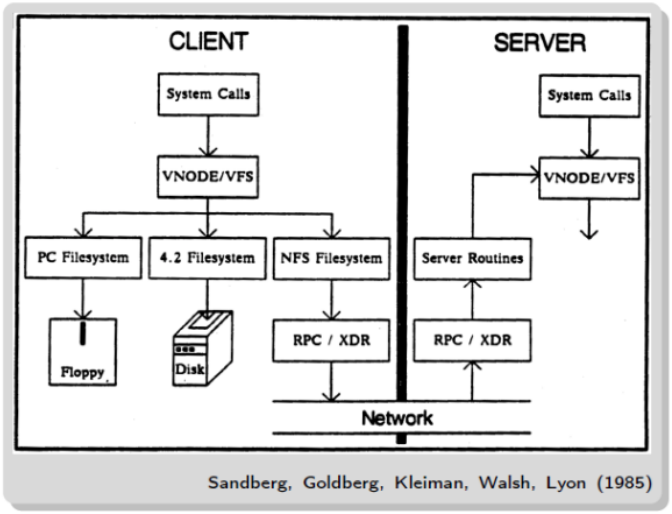
Andrew File System
Client-server

- Roaming home folders
- Identity tokens and access control lists (ACLs)
- Subtree delegation
- Decentralized operation (AFS 3)

补充内容

Milestones in Distributed File Systems





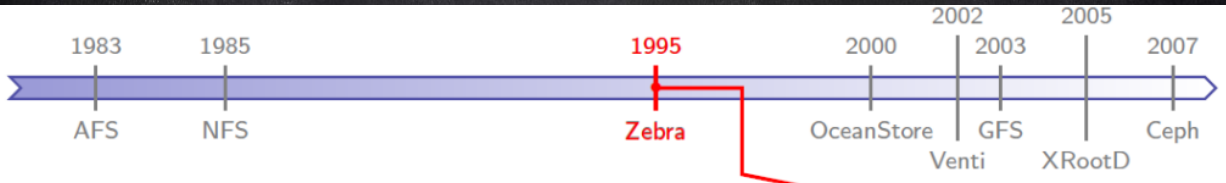
Sandberg, Goldberg, Kleiman, Walsh, Lyon (1985)

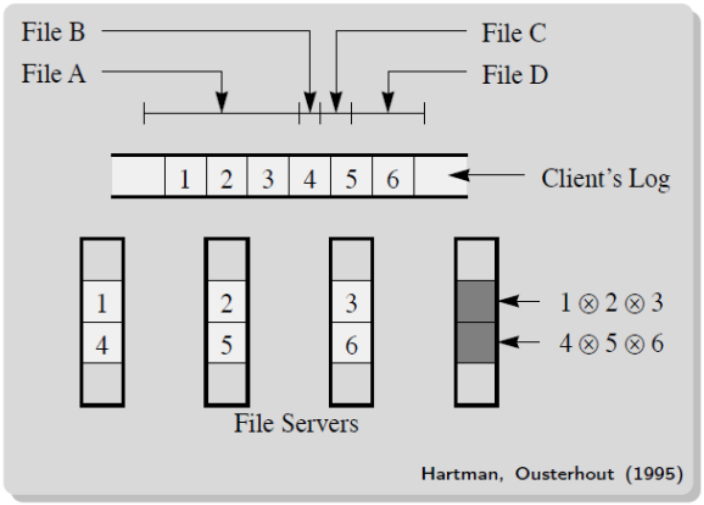
Network File System
Client-server

- Focus on portability
- Separation of protocol and implementation
- Stateless servers
  - Fast crash recovery
  - State re-introduced for locking

补充内容

Milestones in Distributed File Systems



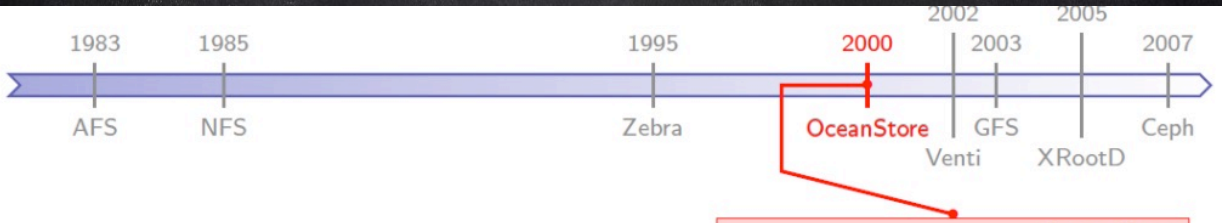


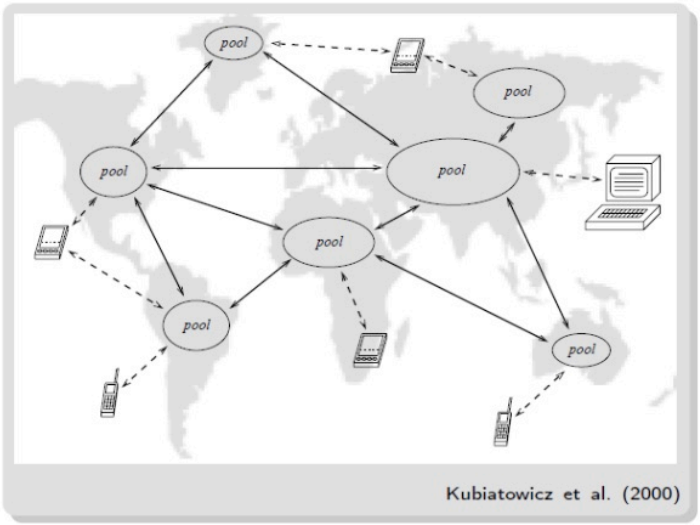
Zebra File System
Parallel

- Striping and parity
- Redundant array of inexpensive nodes (RAIN)
- Log-structured data
- Separation of data and meta-data

补充内容

Milestones in Distributed File Systems



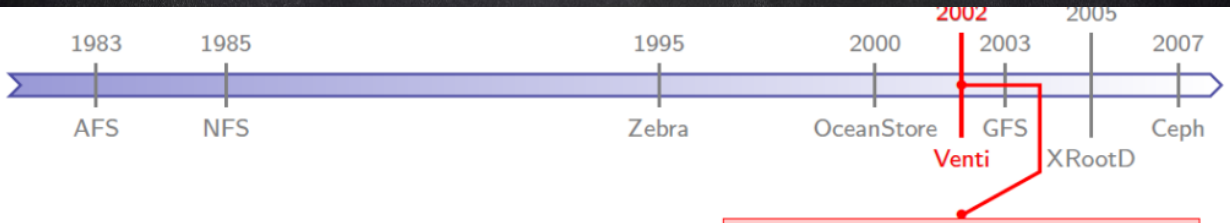


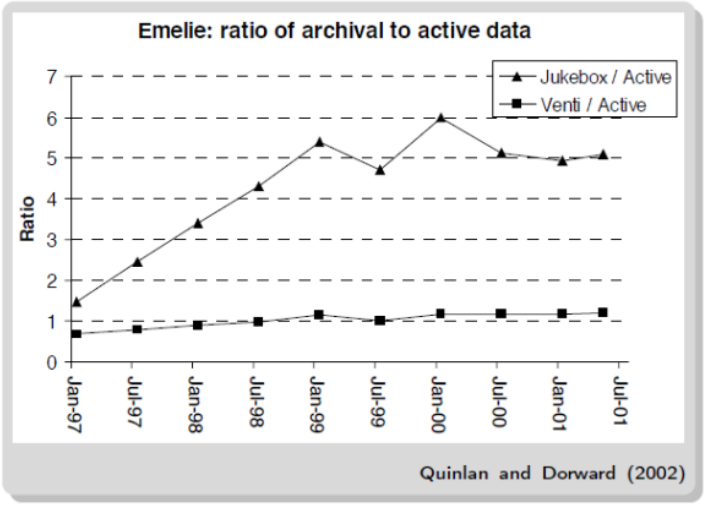
OceanStore
Peer-to-peer

- "Global Scale": 10^{10} users, 10^{14} files
- Untrusted infrastructure
- Based on peer-to-peer overlay network
- Nomadic data through aggressive caching
- Foundation for today's decentral dropbox replacements

补充内容

Milestones in Distributed File Systems





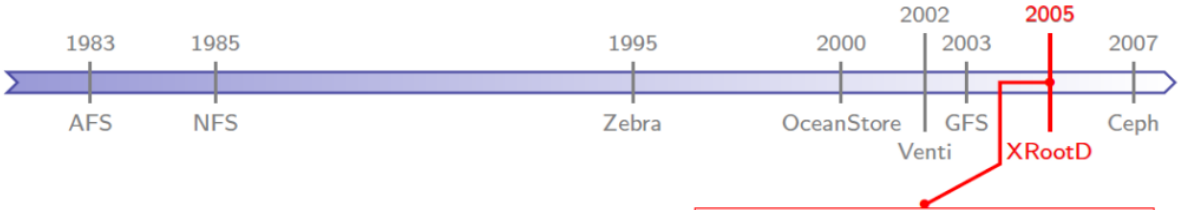
Venti

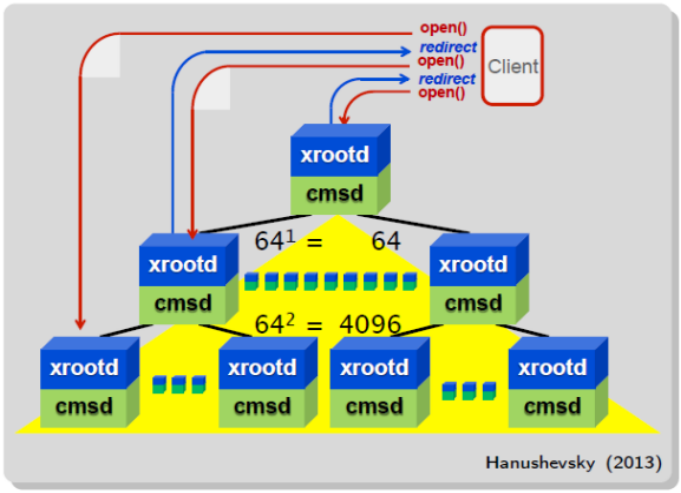
Archival storage

- De-duplication through content-addressable storage
- Content hashes provide intrinsic file integrity
- Merkle trees verify the file system hierarchy

补充内容

Milestones in Distributed File Systems





XRootD

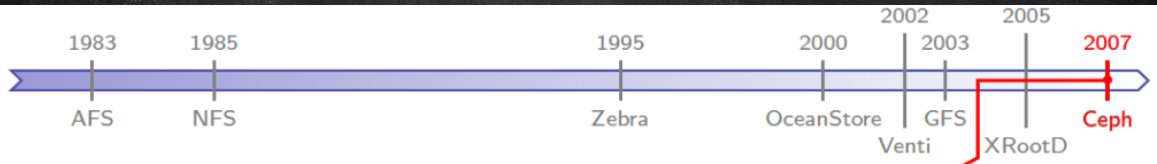
Namespace delegation

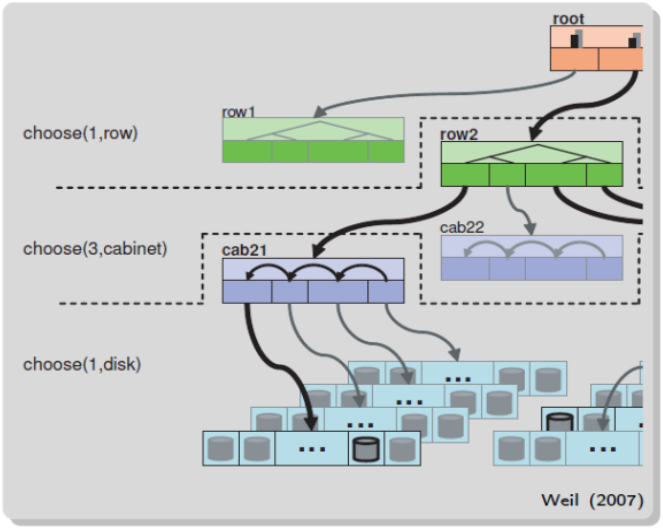
- Global tree of redirectors
- Flexibility through decomposition into pluggable components
- Namespace independent from data access

http://www.xrootd.org/

补充内容

Milestones in Distributed File Systems





Ceph File System and RADOS
Decentralized

- Peer-to-peer file system at the cluster scale
- Incremental scalability through CRUSH distributed hash function
- Adaptive workload distribution

补充内容

What's the next?



补充内容



POSIX Com-pliance

`create()`, `unlink()`, `stat()`	
`read()`, `write()`, `seek()`	
`open()`, `close()`	*essential*
`flock()`	*difficult for DFSs*
`rename()`	
file ownership	
open unlinked files	
hard links	
device files, IPC files	







大数据分析

Motivation: Large Scale Data Processing

- 全社会数据产生的速度非常快



Motivation: Large Scale Data Processing

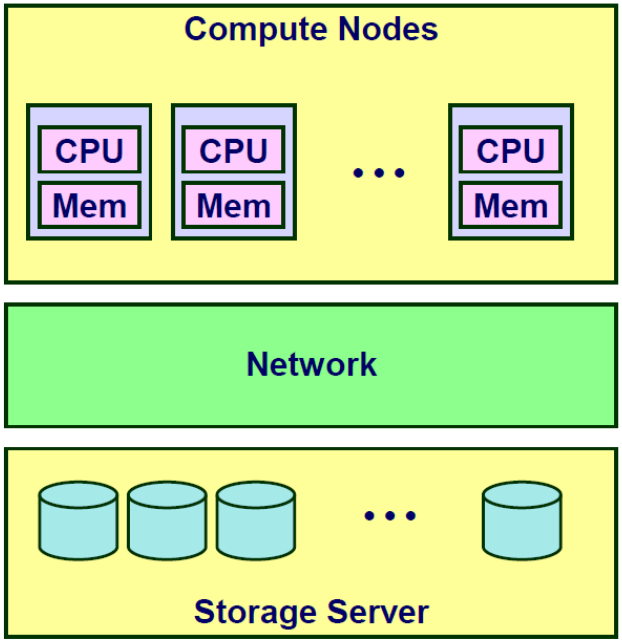
- 大数据的呈现出指数增长速度

全球数据存储量呈现爆炸式增长，企业及互联网数据以每年50%的速率在增长，据Gartner预测，到2020年，全球数据量将达到35ZB，等于80亿块4TB硬盘。





Typical HPC Machine



Compute Nodes

- High end processor(s)
- Lots of RAM

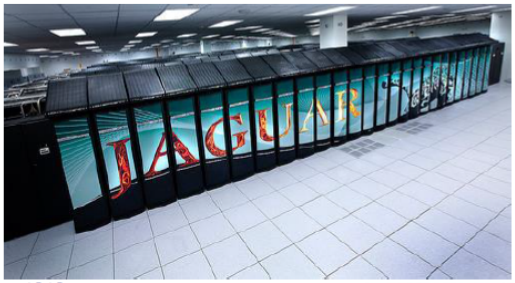
Network

- Specialized
- Very high performance

Storage Server

- RAID-based disk array

HPC Machine Example



Jaguar Supercomputer

- 3rd fastest in world

Compute Nodes

- 18,688 nodes in largest partition
- 2X 2.6Ghz 6-core AMD Opteron
- 16GB memory
- Total: 2.3 petaflop / 300 TB memory



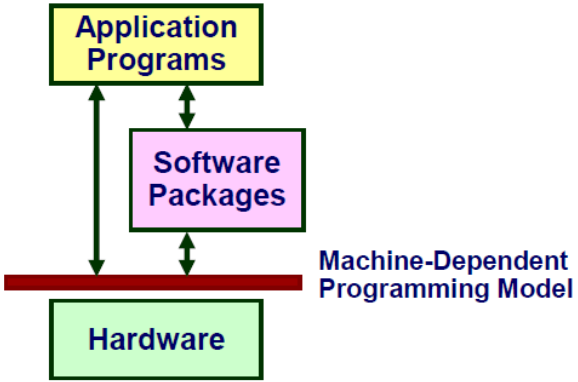
Network

- 3D torus
  - Each node connected to 6 neighbors via 6.0 GB/s links

Storage Server

- 10PB RAID-based disk array

HPC Programming Model



- **Programs described at very low level**
  - Specify detailed control of processing & communications
- **Rely on small number of software packages**
  - Written by specialists
  - Limits classes of problems & solution methods

Bulk Synchronous Programming

Solving Problem Over Grid

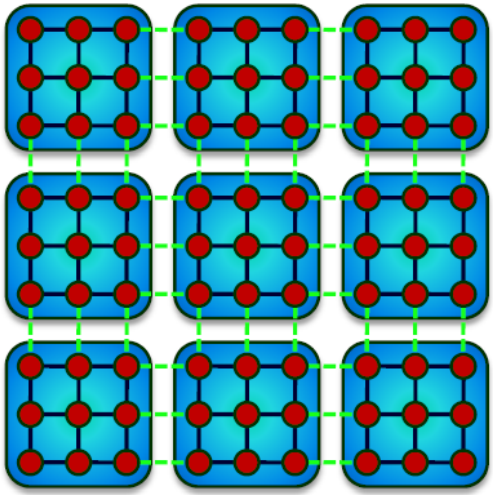
- E.g., finite-element computation

Partition into Regions

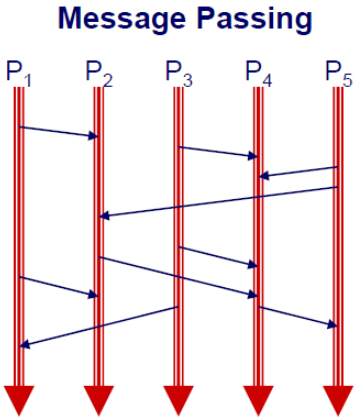
- p regions for p processors

Map Region per Processor

- Local computation sequential
- Periodically communicate boundary values with neighbors



Typical HPC Operation



Characteristics

- Long-lived processes
- Make use of spatial locality
- Hold all program data in memory (no disk access)
- High bandwidth communication

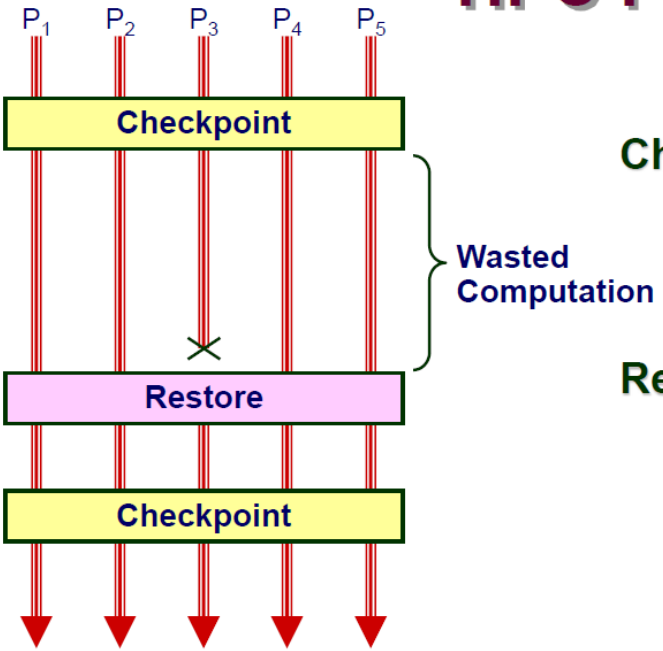
Strengths

- High utilization of resources
- Effective for many scientific applications

Weaknesses

- Requires careful tuning of application to resources
- Intolerant of any variability

HPC Fault Tolerance



Checkpoint

- Periodically store state of all processes
- Significant I/O traffic

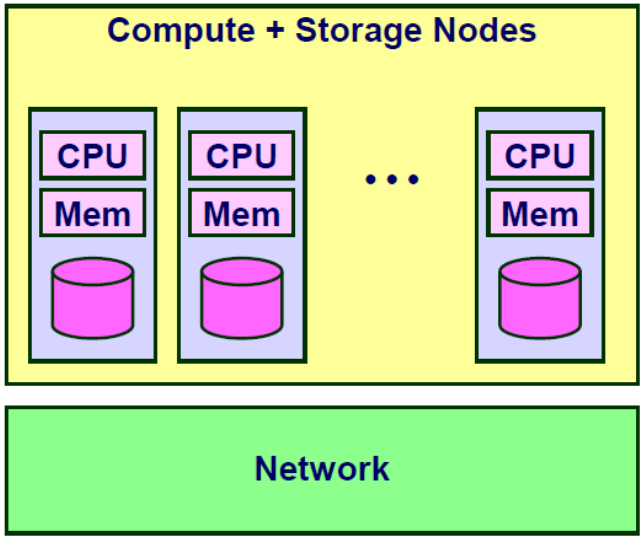
Restore

- When failure occurs
- Reset state to that of last checkpoint
- All intervening computation wasted

Performance Scaling

- Very sensitive to number of failing components

Typical Cluster Machine



Compute + Storage Nodes

- Medium-performance processors
- Modest memory
- 1-2 disks

Network

- Conventional Ethernet switches
  - 10 Gb/s within rack
  - 100 Gb/s across racks

Machines with Disks

Lots of storage for cheap

- **3 TB @ $150**
  (5¢ / GB)
- **Compare 2007:**
  0.75 TB @ $266
  35¢ / GB

Drawbacks

- **Long and highly variable delays**
- **Not very reliable**

Not included in HPC Nodes



Roll over image to zoom in

WD Black 3TB Performance Desktop Hard Disk Drive - 7200 RPM SATA 6 Gb/s 64MB Cache 3.5 Inch - WD3003FZEX

by Western Digital

★★★★½ ▾ 1,615 customer reviews | 28 answered questions

List Price: ~~$249.99~~
Price: $149.99 Prime
You Save: $100.00 (40%)

In Stock.
Ships from and sold by Amazon.com. Gift-wrap available.

Want it Friday, Oct. 30? Order within **55 mins** and choose **Two-Day Shipping** at checkout. Details

Capacity: **3 TB**

1 TB	2 TB	3 TB	4 TB
$71.79 Prime	$116.99 Prime	$149.99 Prime	$198.00 Prime

5 TB	6 TB
$260.99	$298.91

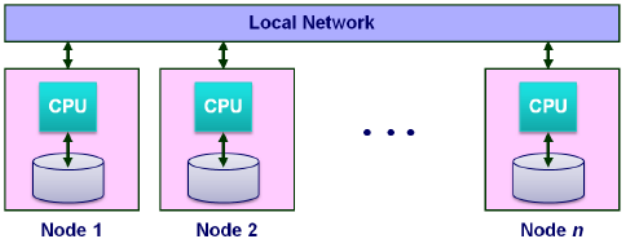
Data-Intensive System Challenge

For Computation That Accesses 1 TB in 5 minutes

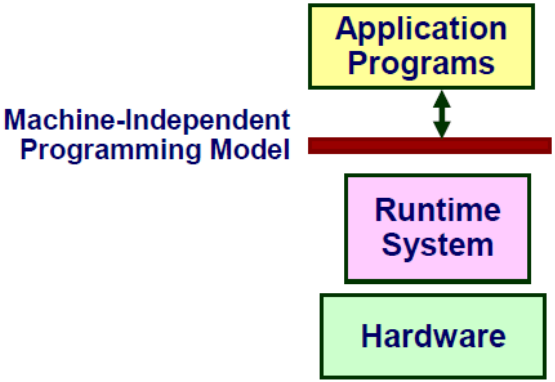
- Data distributed over 100+ disks
  - Assuming uniform data partitioning
- Compute using 100+ processors
- Connected by gigabit Ethernet (or equivalent)

System Requirements

- Lots of disks
- Lots of processors
- Located in close proximity
  - Within reach of fast, local-area network

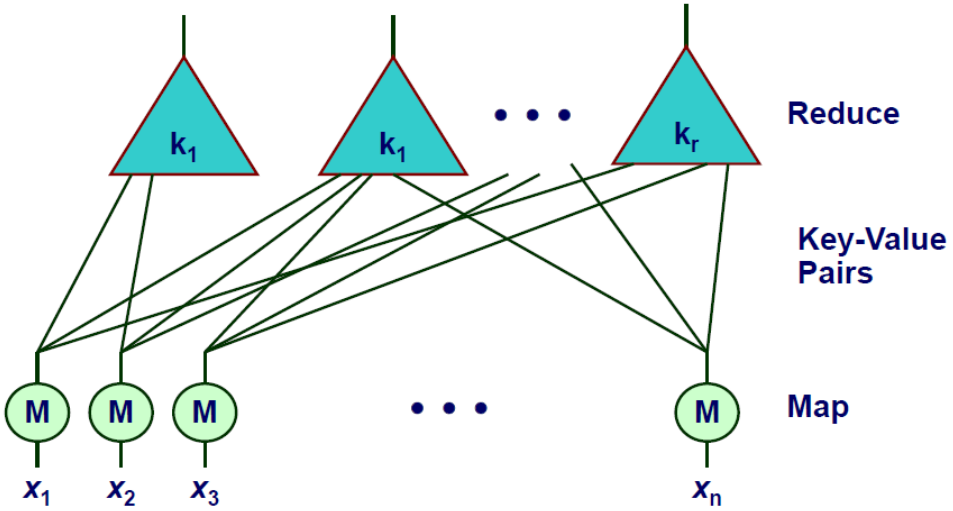


Ideal Cluster Programming Model



- Application programs written in terms of high-level operations on data
- Runtime system controls scheduling, load balancing, …

Map/Reduce Programming Model



- **Map computation across many objects**
  - E.g., 10^{10} Internet web pages
- **Aggregate results in many different ways**
- **System deals with issues of resource allocation & reliability**

Dean & Ghemawat: "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004

MapReduce Example



Come, Dick

Come and see.

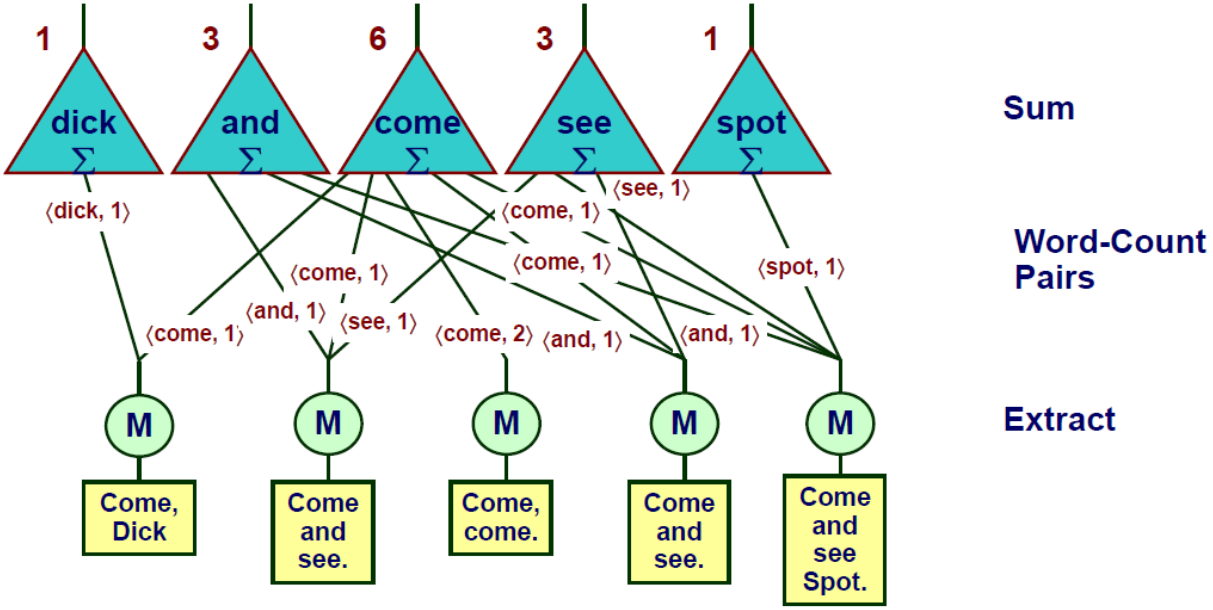
Come, come.

Come and see.

Come and see Spot.

- Create an word index of set of documents

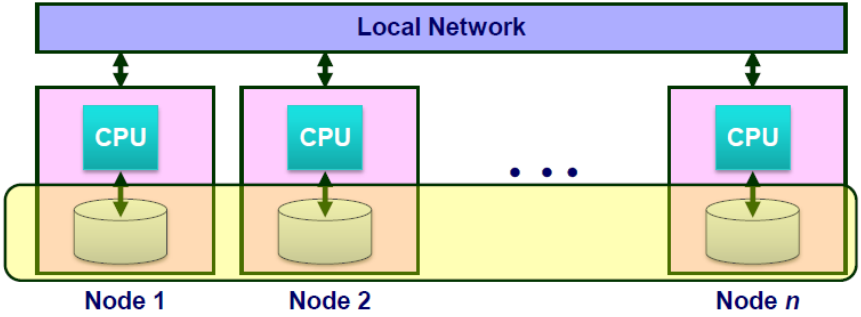
MapReduce Example



- Map: generate ⟨word, count⟩ pairs for all words in document
- Reduce: sum word counts across documents

Hadoop Project

File system with files distributed across nodes



- **Store multiple (typically 3 copies of each file)**
  - If one node fails, data still available
- **Logically, any node has access to any file**
  - May need to fetch across network (ideally, leverage locality for perf.)

Map / Reduce programming environment

- **Software manages execution of tasks on nodes**





Hadoop MapReduce API

Requirements

- Programmer must supply Mapper & Reducer classes

Mapper

- Steps through file one line at a time
- Code generates sequence of <key, value> pairs
  - Call output.collect(key, value)
- Default types for keys & values are strings
  - Lots of low-level machinery to convert to & from other data types
  - But can use anything “writable”

Reducer

- Given key + iterator that generates sequence of values
- Generate one or more <key, value> pairs
  - Call output.collect(key, value)





Cluster Scalability Advantages

- Distributed system design principles lead to scalable design
- Dynamically scheduled tasks with state held in replicated files

Provisioning Advantages

- Can use consumer-grade components
  - maximizes cost-peformance
- Can have heterogenous nodes
  - More efficient technology refresh

Operational Advantages

- Minimal staffing
- No downtime





Example: Sparse Matrices with Map/Reduce

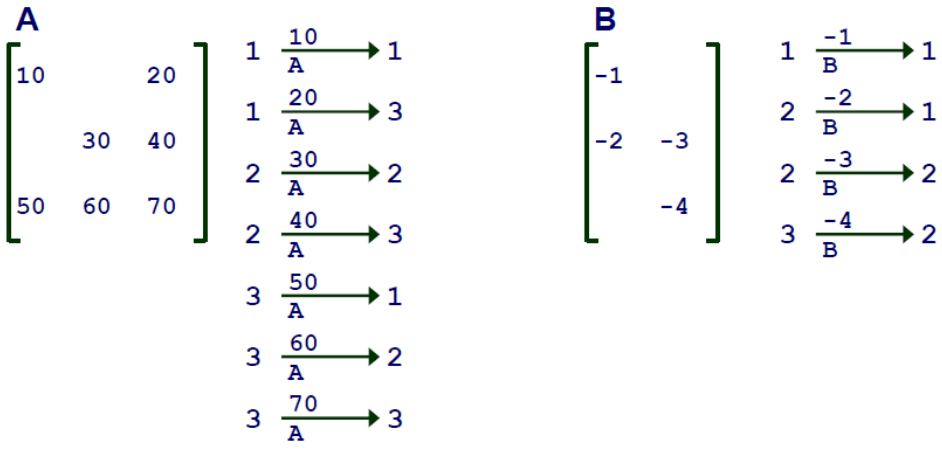
$$A = \begin{bmatrix} 10 & & 20 \\ & 30 & 40 \\ 50 & 60 & 70 \end{bmatrix} \quad X \quad B = \begin{bmatrix} -1 & \\ -2 & -3 \\ & -4 \end{bmatrix} \quad = \quad C = \begin{bmatrix} -10 & -80 \\ -60 & -250 \\ -170 & -460 \end{bmatrix}$$

- Task: Compute product C = A·B
- Assume most matrix entries are 0

Motivation

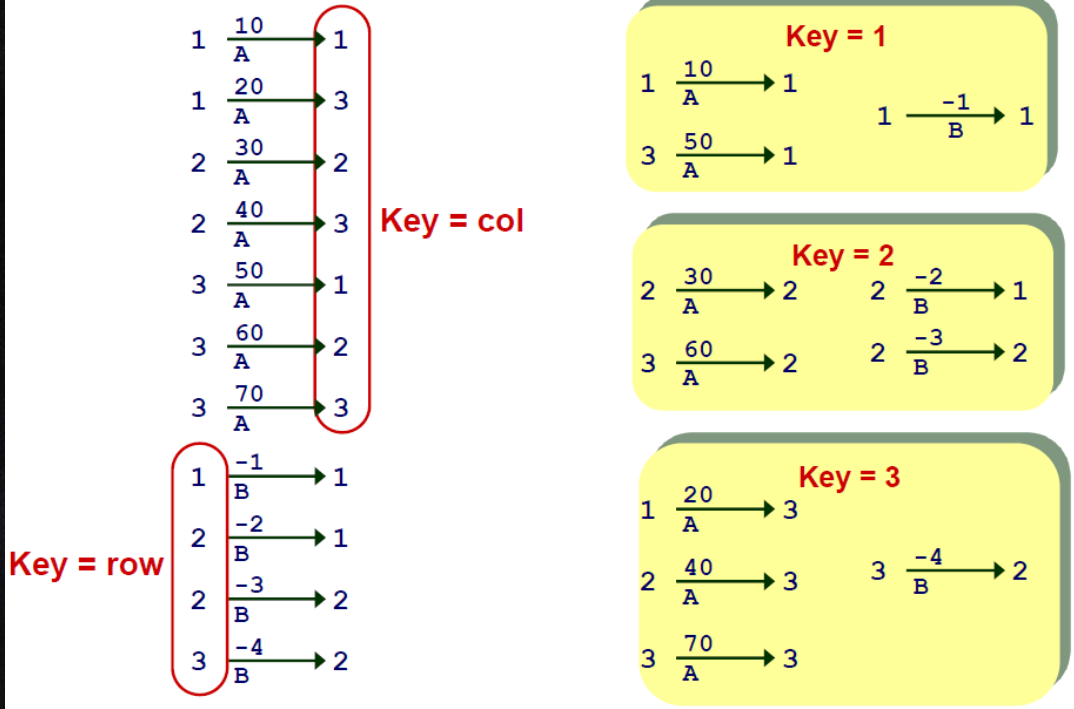
- Core problem in scientific computing
- Challenging for parallel execution
- Demonstrate expressiveness of Map/Reduce

Computing Sparse Matrix Product



- Represent matrix as list of nonzero entries
  ⟨row, col, value, matrixID⟩
- Strategy
  - Phase 1: Compute all products $a_{i,k} \cdot b_{k,j}$
  - Phase 2: Sum products for each entry i,j
  - Each phase involves a Map/Reduce

Phase 1 Map of Matrix Multiply



- Group values $a_{i,k}$ and $b_{k,j}$ according to key k

Phase 1 "Reduce" of Matrix Multiply

Key = 1

1 —10, A→ 1
3 —50, A→ 1

X

1 —-1, B→ 1

Key = 2

2 —30, A→ 2
3 —60, A→ 2

X

2 —-2, B→ 1
2 —-3, B→ 2

Key = 3

1 —20, A→ 3
2 —40, A→ 3
3 —70, A→ 3

X

3 —-4, B→ 2

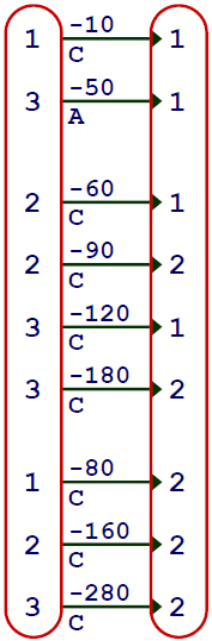
1 —-10, C→ 1
3 —-50, A→ 1

2 —-60, C→ 1
2 —-90, C→ 2
3 —-120, C→ 1
3 —-180, C→ 2

1 —-80, C→ 2
2 —-160, C→ 2
3 —-280, C→ 2

- Generate all products $a_{i,k} \cdot b_{k,j}$

Phase 2 Map of Matrix Multiply



Key = row,col

Key = 1,1 — 1 -10 C → 1

Key = 1,2 — 1 -80 C → 2

Key = 2,1 — 2 -60 C → 1

Key = 2,2 — 2 -90 C → 2; 2 -160 C → 2

Key = 3,1 — 3 -120 C → 1; 3 -50 A → 1

Key = 3,2 — 3 -280 C → 2; 3 -180 C → 2

- Group products $a_{i,k} \cdot b_{k,j}$ with matching values of i and j

Phase 2 Reduce of Matrix Multiply

Key = 1,1	1 —(-10, C)→ 1	1 —(-10, C)→ 1
Key = 1,2	1 —(-80, C)→ 2	1 —(-80, C)→ 2
Key = 2,1	2 —(-60, C)→ 1	2 —(-60, C)→ 1
Key = 2,2	2 —(-90, C)→ 2 2 —(-160, C)→ 2	2 —(-250, C)→ 2
Key = 3,1	3 —(-120, C)→ 1 3 —(-50, A)→ 1	3 —(-170, C)→ 1
Key = 3,2	3 —(-280, C)→ 2 3 —(-180, C)→ 2	3 —(-460, C)→ 2

C

$$\begin{bmatrix} -10 & -80 \\ -60 & -250 \\ -170 & -460 \end{bmatrix}$$

- Sum products to get final entries





Matrix Multiply Phase 1 Mapper

```
public class P1Mapper extends MapReduceBase implements Mapper {

    public void map(WritableComparable key, Writable values,
                    OutputCollector output, Reporter reporter) throws
IOException {
        try {
            GraphEdge e = new GraphEdge(values.toString());
            IntWritable k;
            if (e.tag.equals("A"))
                k = new IntWritable(e.toNode);
            else
                k = new IntWritable(e.fromNode);
            output.collect(k, new Text(e.toString()));
        } catch (BadGraphException e) {}
    }
}
```





Matrix Multiply Phase 1 Reducer

```
public class P1Reducer extends MapReduceBase implements Reducer {

        public void reduce(WritableComparable key, Iterator values,
                           OutputCollector output, Reporter reporter)
                           throws IOException
    {
        Text outv = new Text(""); // Don't really need output values
        /* First split edges into A and B categories */
        LinkedList<GraphEdge> alist = new LinkedList<GraphEdge>();
        LinkedList<GraphEdge> blist = new LinkedList<GraphEdge>();
        while(values.hasNext()) {
                try {
                    GraphEdge e =
                        new GraphEdge(values.next().toString());
                    if (e.tag.equals("A")) {
                        alist.add(e);
                    } else {
                        blist.add(e);
                    }
                } catch (BadGraphException e) {}
        }
        // Continued
```





MM Phase 1 Reducer (cont.)

```
            // Continuation

            Iterator<GraphEdge> aset = alist.iterator();
            // For each incoming edge
            while(aset.hasNext()) {
                GraphEdge aedge = aset.next();
                // For each outgoing edge
                Iterator<GraphEdge> bset = blist.iterator();
                while (bset.hasNext()) {
                    GraphEdge bedge = bset.next();
                    GraphEdge newe = aedge.contractProd(bedge);
                    // Null would indicate invalid contraction
                    if (newe != null) {
                        Text outk = new Text(newe.toString());
                        output.collect(outk, outv);
                    }
                }
            }
        }
}
```





Matrix Multiply Phase 2 Mapper

```
public class P2Mapper extends MapReduceBase implements Mapper {

    public void map(WritableComparable key, Writable values,
                    OutputCollector output, Reporter reporter)
                        throws IOException {
        String es = values.toString();
        try {
            GraphEdge e = new GraphEdge(es);
            // Key based on head & tail nodes
            String ks = e.fromNode + " " + e.toNode;
            output.collect(new Text(ks), new Text(e.toString()));
        } catch (BadGraphException e) {}

    }
}
```





Matrix Multiply Phase 2 Reducer

```
public class P2Reducer extends MapReduceBase implements Reducer {

        public void reduce(WritableComparable key, Iterator values,
                           OutputCollector output, Reporter reporter)
                                throws IOException
    {
        GraphEdge efinal = null;
        while (efinal == null && values.hasNext()) {
            try {
                efinal = new GraphEdge(values.next().toString());
            } catch (BadGraphException e) {}
        }
        if (efinal != null) {
            while(values.hasNext()) {
                try {
                    GraphEdge eother =
                        new GraphEdge(values.next().toString());
                    efinal.weight += eother.weight;
                } catch (BadGraphException e) {}
            }
            if (efinal.weight != 0)
                output.collect(new Text(efinal.toString()),
                        new Text(""));
        }
    }
}
```







Lessons from Sparse Matrix Example

Associative Matching is Powerful Communication Primitive

- Intermediate step in Map/Reduce

Similar Strategy Applies to Other Problems

- Shortest path in graph
- Database join

Many Performance Considerations

- Kiefer, Volk, Lehner, TU Dresden
- Should do systematic comparison to other sparse matrix implementations





MapReduce Implementation

Built on Top of Parallel File System

- Google: GFS, Hadoop: HDFS
- Provides global naming
- Reliability via replication (typically 3 copies)

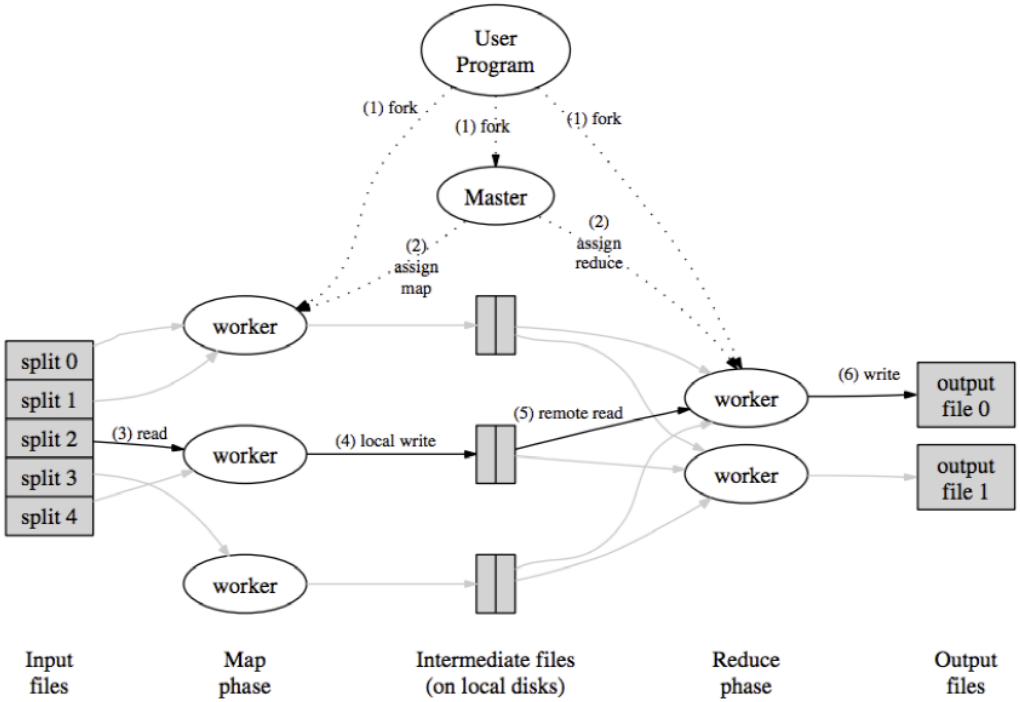
Breaks work into tasks

- Master schedules tasks on workers dynamically
- Typically #tasks >> #processors

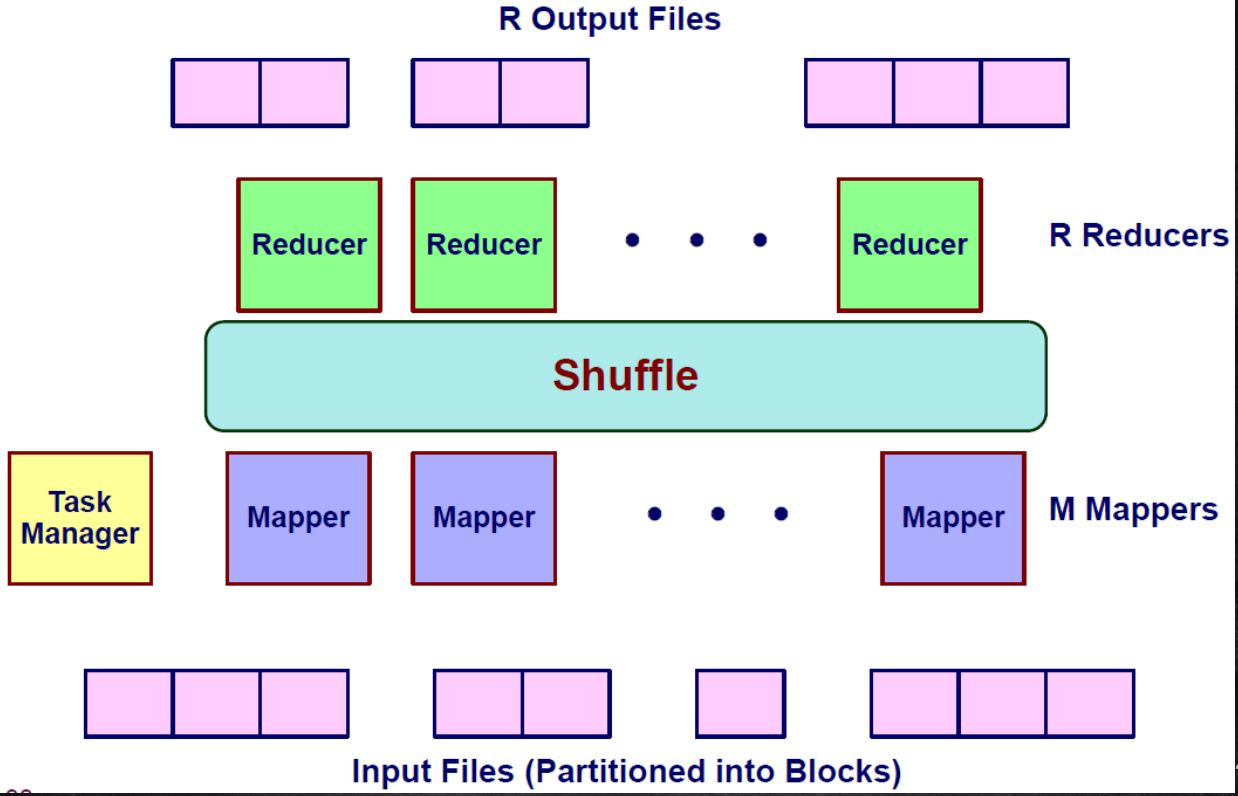
Net Effect

- Input: Set of files in reliable file system
- Output: Set of files in reliable file system

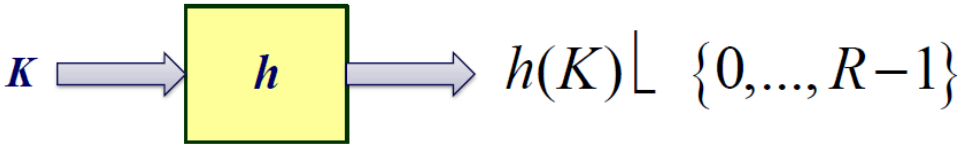
MapReduce Execution (authors view)



MapReduce Execution



Mapping

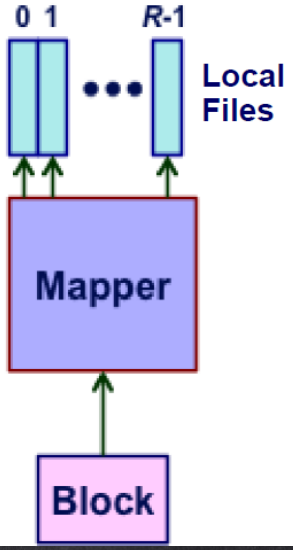


Hash Function *h*

- Maps each key *K* to integer *i* such that $0 \le i < R$

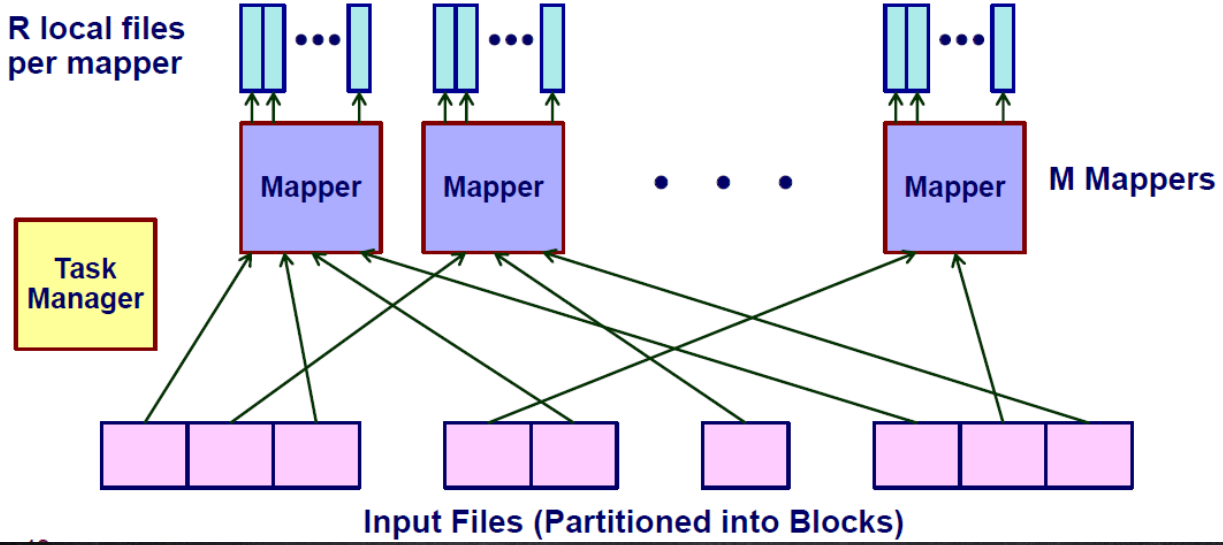
Mapper Operation

- Reads input file blocks
- Generates pairs ⟨*K*, *V*⟩
- Writes to local file *h*(*K*)



Mapping

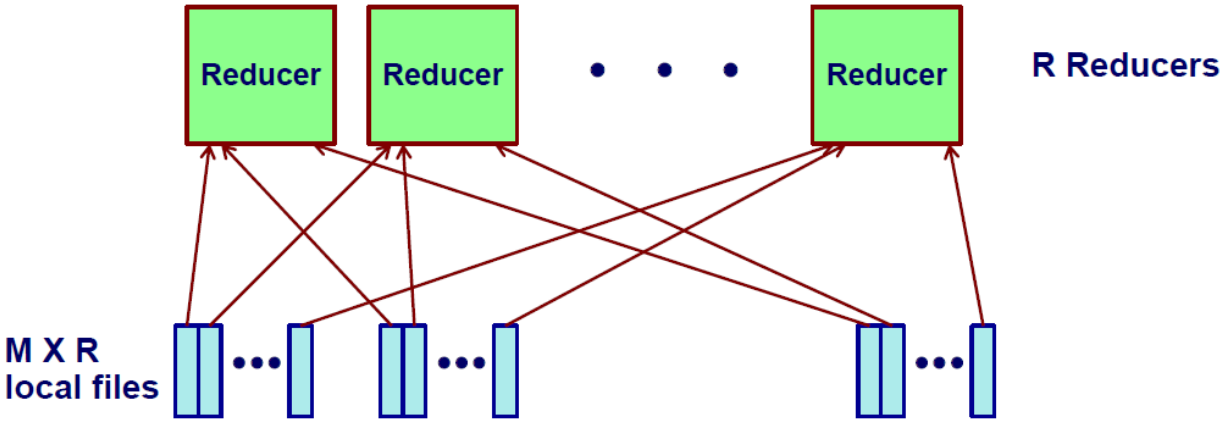
- Dynamically map input file blocks onto mappers
- Each generates key/value pairs from its blocks
- Each writes R files on local file system



Shuffling

Each Reducer:

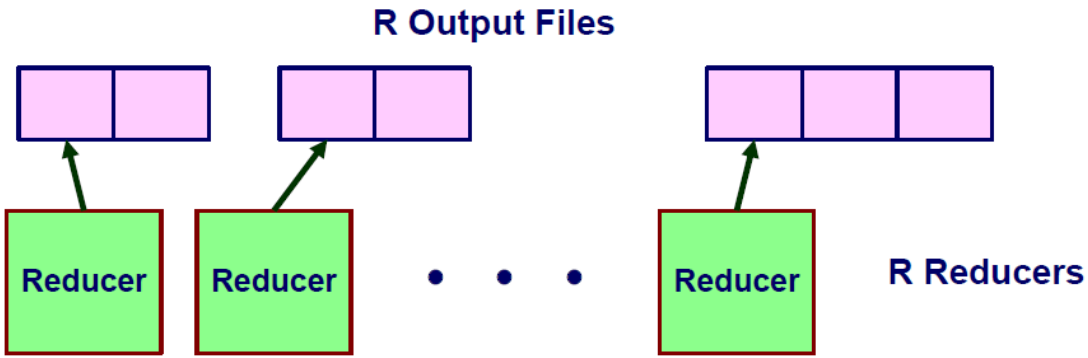
- Handles 1/R of the possible key values
- Fetches its file from each of M mappers
- Sorts all of its entries to group values by keys



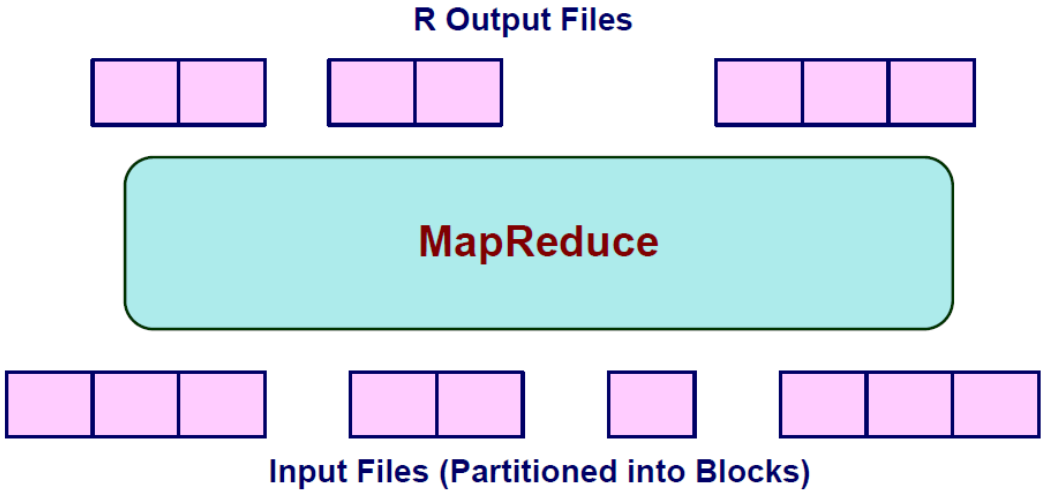
Reducing

Each Reducer:

- Executes reducer function for each key
- Writes output values to parallel file system



MapReduce Effect

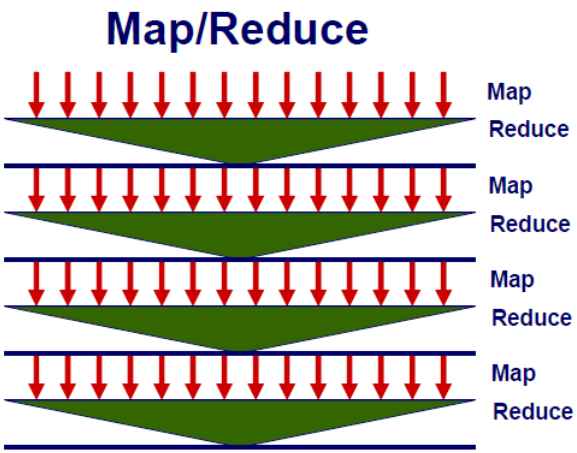


MapReduce Step

- Reads set of files from file system
- Generates new set of files

Can iterate to do more complex processing

Map/Reduce Operation



Characteristics

- Computation broken into many, short-lived tasks
  - Mapping, reducing
- Use disk storage to hold intermediate results

Strengths

- Great flexibility in placement, scheduling, and load balancing
- Can access large data sets

Weaknesses

- Higher overhead
- Lower raw performance





Example Parameters

Sort Benchmark

- 10^{10} 100-byte records
- Partition into M = 15,000 64MB pieces
  - Key = value
  - Partition according to most significant bytes
- Sort locally with R = 4,000 reducers

Machine

- 1800 2Ghz Xeons
- Each with 2 160GB IDE disks
- Gigabit ethernet
- 891 seconds total





Interesting Features

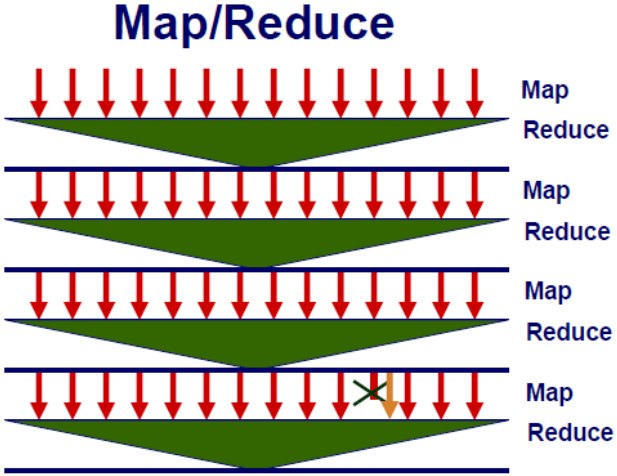
Fault Tolerance

- Assume reliable file system
- Detect failed worker
  - Heartbeat mechanism
- Reschedule failed task

Stragglers

- Tasks that take long time to execute
- Might be bug, flaky hardware, or poor partitioning
- When done with most tasks, reschedule any remaining executing tasks
  - Keep track of redundant executions
  - Significantly reduces overall run time

Map/Reduce Fault Tolerance



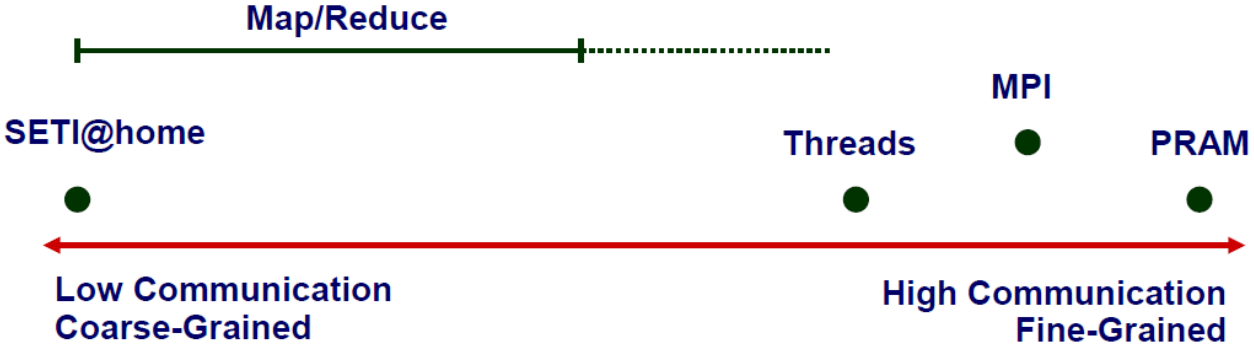
Data Integrity

- Store multiple copies of each file
- Including intermediate results of each Map / Reduce
  - Continuous checkpointing

Recovering from Failure

- Simply recompute lost result
  - Localized effect
- Dynamic scheduler keeps all processors busy

Exploring Parallel Computation Models



Map/Reduce Provides Coarse-Grained Parallelism

- Computation done by independent processes
- File-based communication

Observations

- Relatively “natural” programming model
- Research issue to explore full potential and limits





Beyond Map/Reduce

Typical Map/Reduce Applications

- Sequence of steps, each requiring map & reduce
- Series of data transformations
- Iterating until reach convergence

Strengths of Map/Reduce

- User writes simple functions, system manages complexities of mapping, synchronization, fault tolerance
- Very general
- Good for large-scale data analysis

Limitations

- No locality of data or activity
- Each map/reduce step must complete before next begins





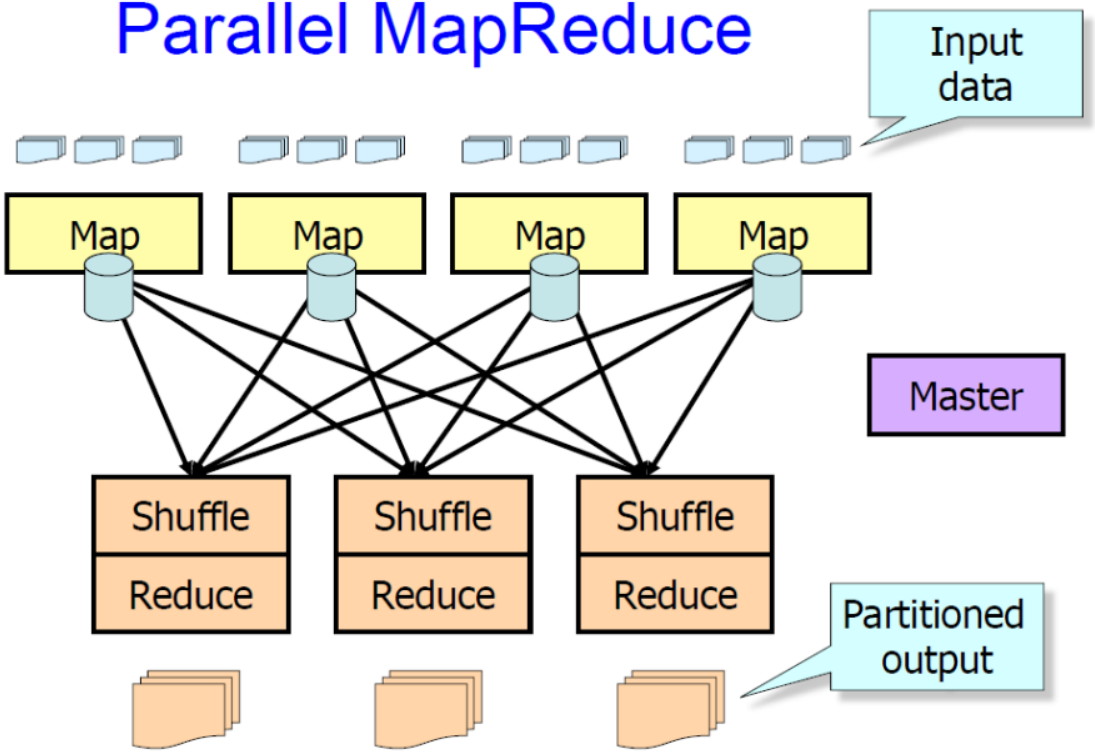


分布式机器学习





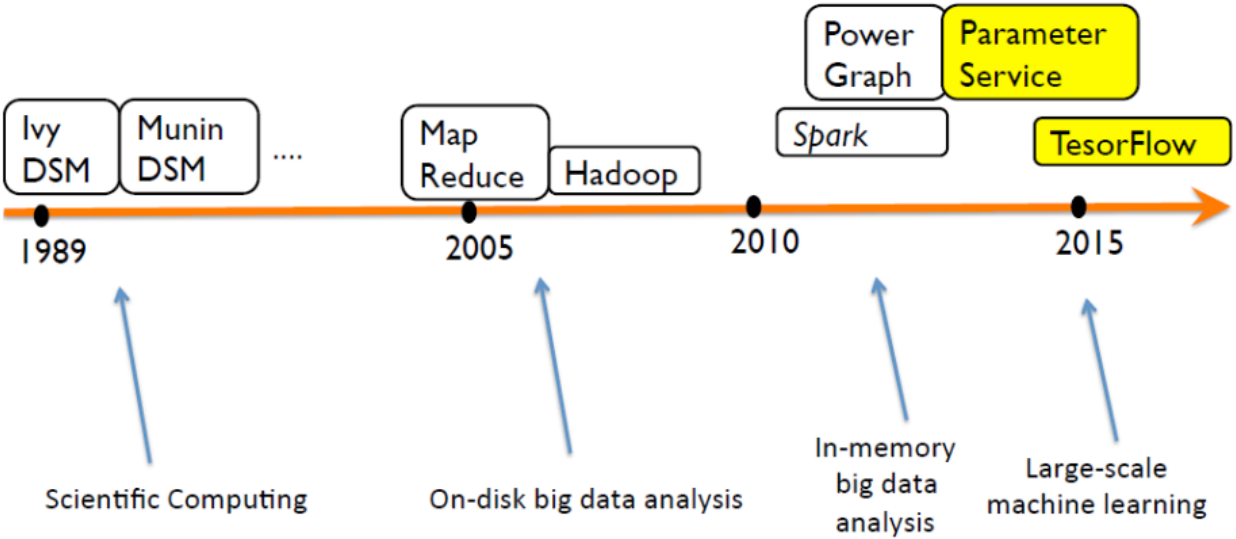
Parallel MapReduce







Distributed Computation







Large-scale Machine learning

- Applying ML algorithms to large quantities of data
- More training data → Better ML predictions
- More training data → Expensive computation → needs acceleration via distribution
- e.g.
  - cluster all news articles into different categories
  - recommending products/movies/articles to users
  - vision, machine translation, sentiment analysis





Some ML algorithms work well with MapReduce/Spark

- Example: k-means
  - Given data points (x_1, x_2, ..., x_n), partition them into k clusters with centroids (u_1, u_2, ..., u_k), such that the distances to assigned centroids are minimized
- Algorithm:
  - initialize random k centroids
  - for each data point, assign it to the ``closest'' centroid
  - for each cluster, compute the average of all assigned points as new centroid
  - iterate





```
...
while tempDist > convergeDist:

    closest = data.map(
        lambda p: (closestPoint(p, kPoints), (p, 1)))

    pointStats = closest.reduceByKey(
        lambda p1_c1, p2_c2: (p1_c1[0] + p2_c2[0], p1_c1[1] + p2_c2[1]))

    newPoints = pointStats.map(
        lambda st: (st[0], st[1][0] / st[1][1])).collect()

    tempDist = sum(numpy.sum((kPoint[iK] - p

    for (iK, p) in newPoints:
        kPoints[iK] = p
```

For each point, output the index of the best centroid , p, 1(used as count)

For each cluster, add point coordinates and counts together

For each cluster, scale summed point coordinates by counts to compute new centroids

Turn RDD into a normal variable that will be copied to every worker

https://github.com/a amples/src/main/python/kmeans.py



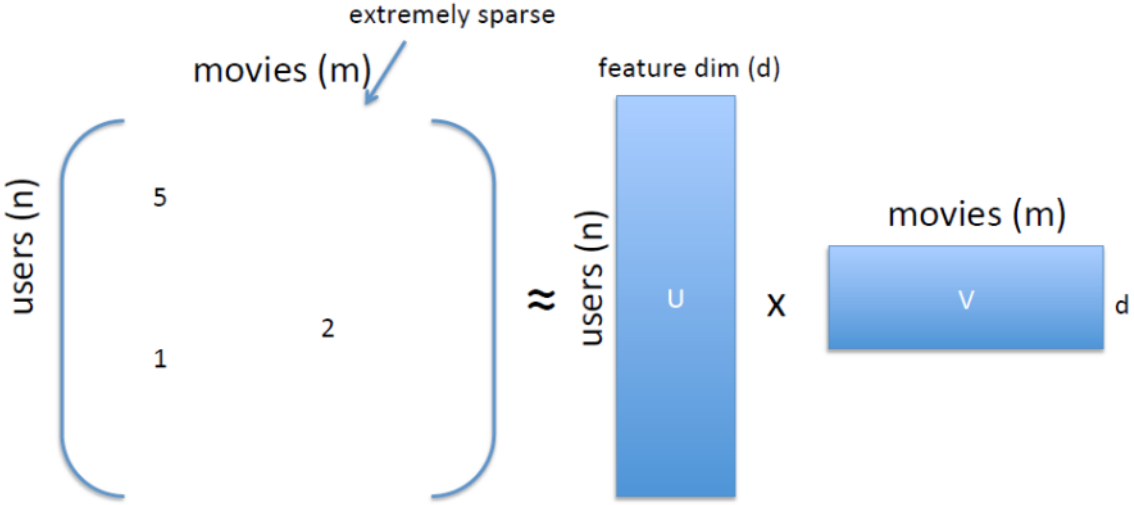


Some ML algorithms work well with MapReduce/Spark

- Why k-means work well with MapReduce/Spark?
- Small parameters copied to and from master
  - k centroids
- Each iteration scans large amounts of data

Some ML algorithm works well with Graph frameworks

- Example: Collaborative filtering via ALS



Find U and V such that the difference between R and U*V are minimized

- ALS: Alternating Minimum Squares
  - Fix U, optimize V, then fix V, optimize U etc.

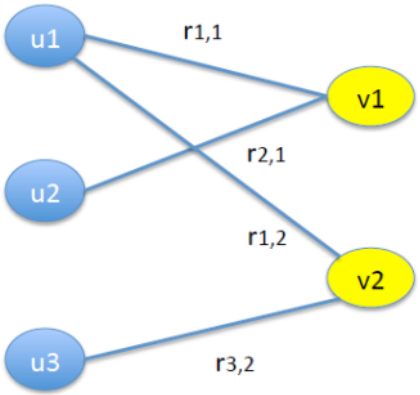
repeat

For i in 1, 2, ..., n

$$u_i = (\sum_{r_{i,j}} v_j v_j^T)^{-1} \sum_{r_{i,j}} r_{i,j} v_j$$

For j in 1, 2, ..., n

$$v_j = (\sum_{r_{i,j}} u_i u_i^T)^{-1} \sum_{r_{i,j}} r_{i,j} u_i$$

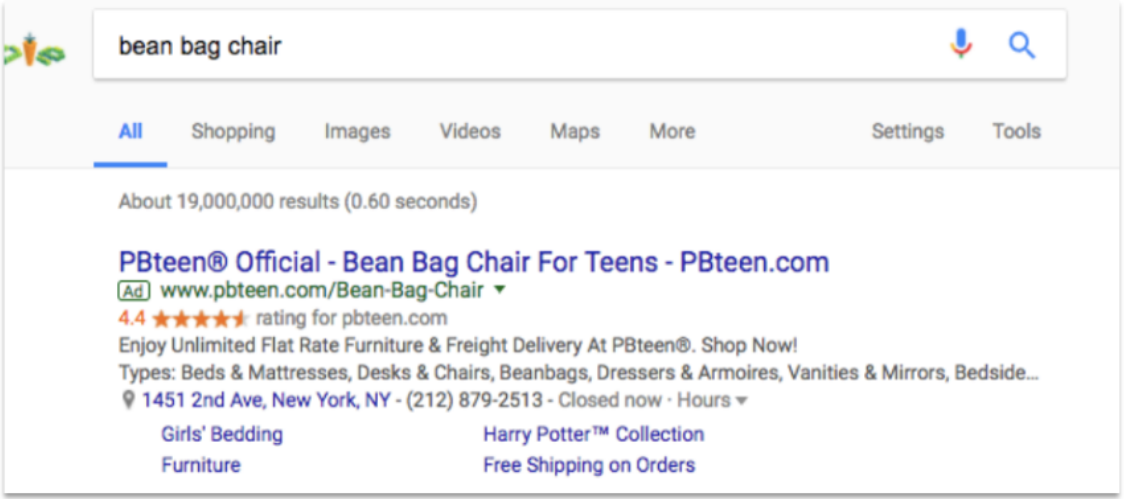


*I did not include regularization terms

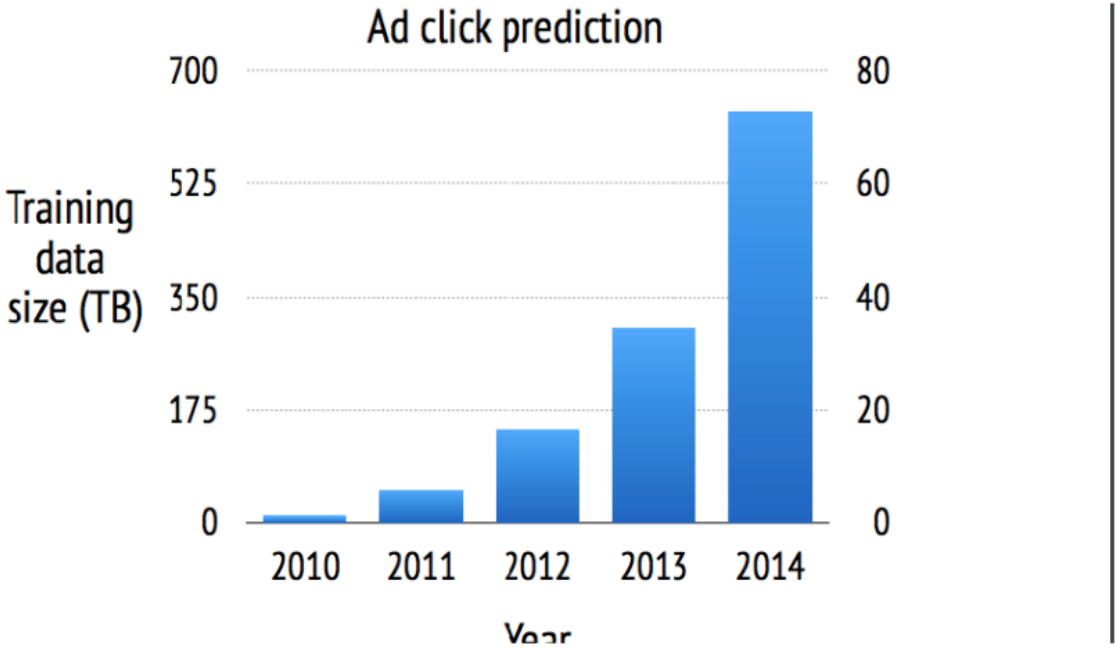
ALS could be written in Spark too using joins

Beyond MapReduce and Graph Computing

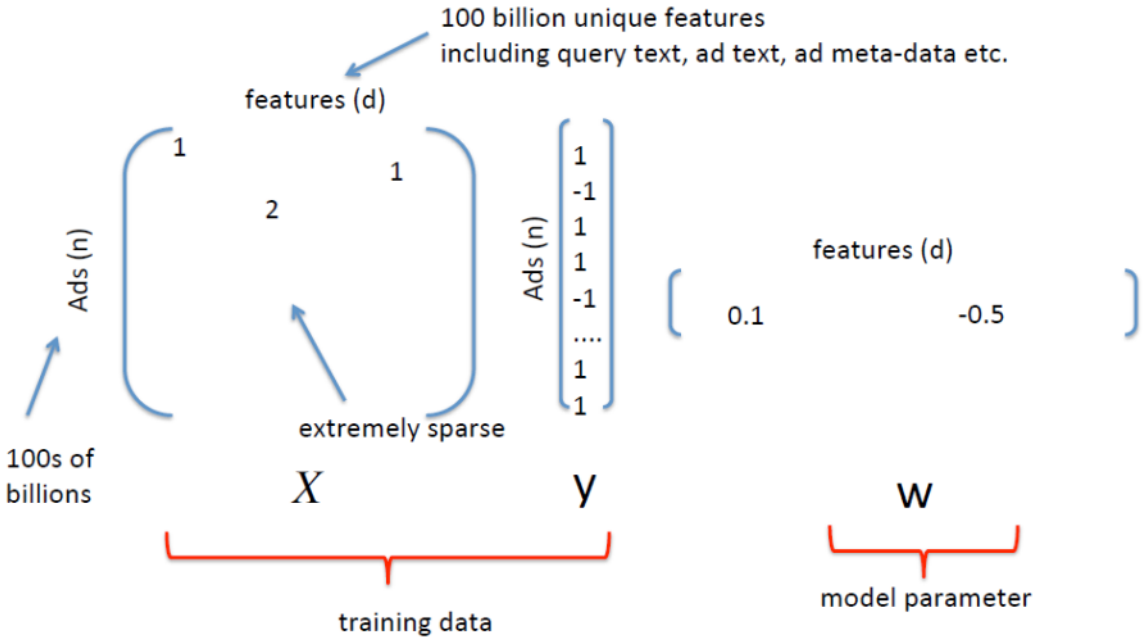
- Example large-scale ML problem: predict whether a user will click an ad



A very large-scale ML problem



Ad-click prediction w/ logistic regression

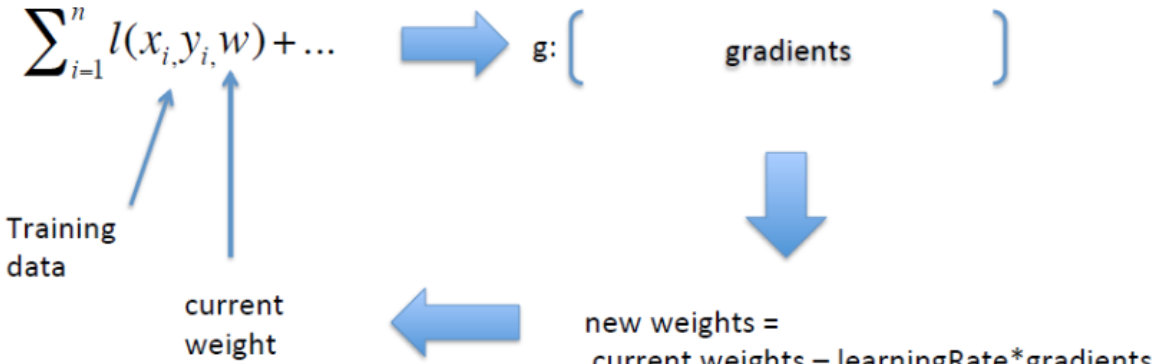


Predicted clickiness: $f(x \cdot w^T)$

Find w that minimizes loss with regulization: $\sum_{i=1}^{n} l(x_i, y_i, w) + \ldots$

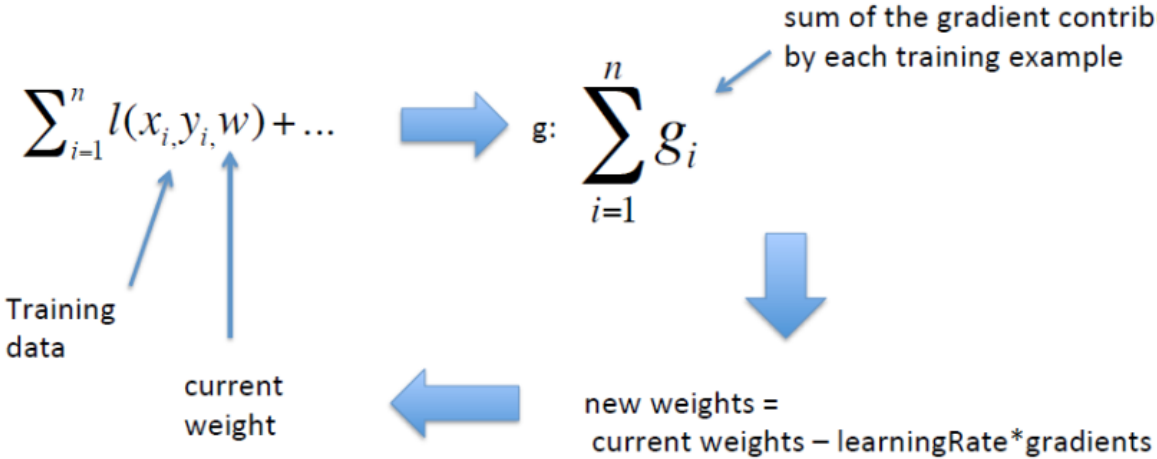
Solving logistic regression

- Batch gradient descent

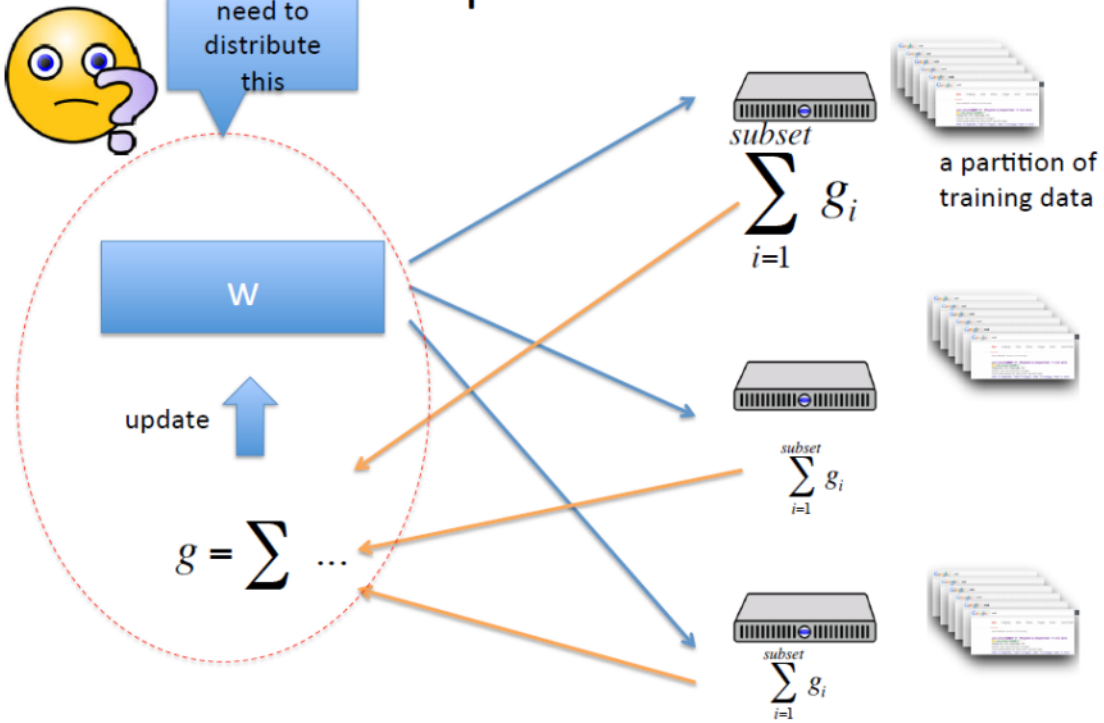


Solving logistic regression

- Batch gradient descent



Distributing logistic regression: data parallelism







Parameter Server [OSDI'14]

Scaling Distributed Machine Learning with the Parameter Server

Mu Li*‡, David G. Andersen*, Jun Woo Park*, Alexander J. Smola*†, Amr Ahmed†,
Vanja Josifovski†, James Long†, Eugene J. Shekita†, Bor-Yiing Su†

*Carnegie Mellon University ‡Baidu †Google

{muli, dga, junwoop}@cs.cmu.edu, alex@smola.org, {amra, vanjaj, jamlong, shekita, boryiingsu}@google.com

Abstract

We propose a parameter server framework for distributed machine learning problems. Both data and workloads are distributed over worker nodes, while the server nodes maintain globally shared parameters, represented as dense or sparse vectors and matrices. The framework manages asynchronous data communication between nodes, and supports flexible consistency models, elastic scalability, and continuous fault tolerance.

To demonstrate the scalability of the proposed framework, we show experimental results on petabytes of real data with billions of examples and parameters on problems ranging from Sparse Logistic Regression to Latent Dirichlet Allocation and Distributed Sketching.

≈ #machine × time	# of jobs	failure rate
100 hours	13,187	7.8%
1,000 hours	1,366	13.7%
10,000 hours	77	24.7%

Table 1: Statistics of machine learning jobs for a three month period in a data center.

cost of synchronization and machine latency is high.

- At scale, fault tolerance is critical. Learning tasks are often performed in a cloud environment where machines can be unreliable and jobs can be preempted.

To illustrate the last point, we collected all job logs for

Key problems

1.Accessing the parameters requires an enormous amount of network bandwidth.

2.Many machine learning algorithms are sequential.

3.At scale, fault tolerance is critical.

	Shared Data	Consistency	Fault Tolerance
Graphlab [34]	graph	eventual	checkpoint
Petuum [12]	hash table	delay bound	none
REEF [10]	array	BSP	checkpoint
Naiad [37]	(key,value)	multiple	checkpoint
Mlbase [29]	table	BSP	RDD
Parameter Server	(sparse) vector/matrix	various	continuous

Table 2: Attributes of distributed data analysis systems.

Architecture

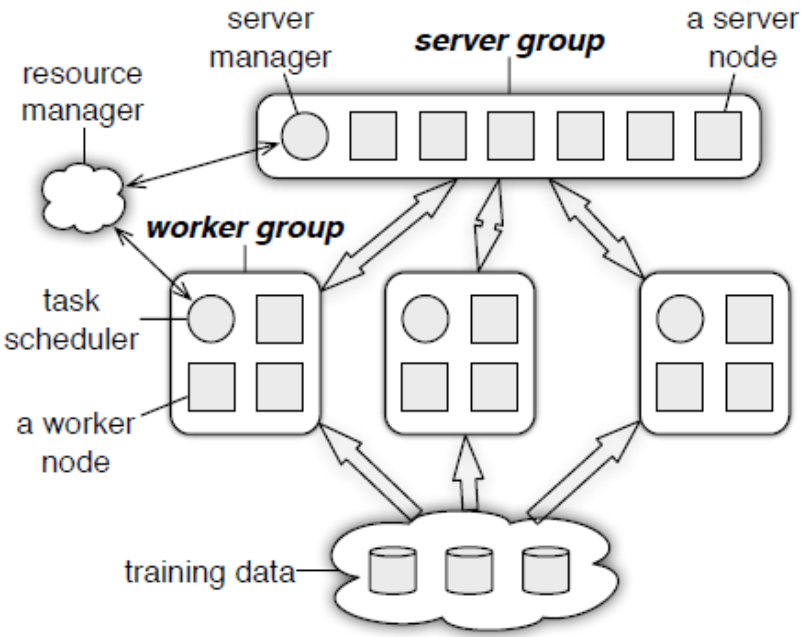
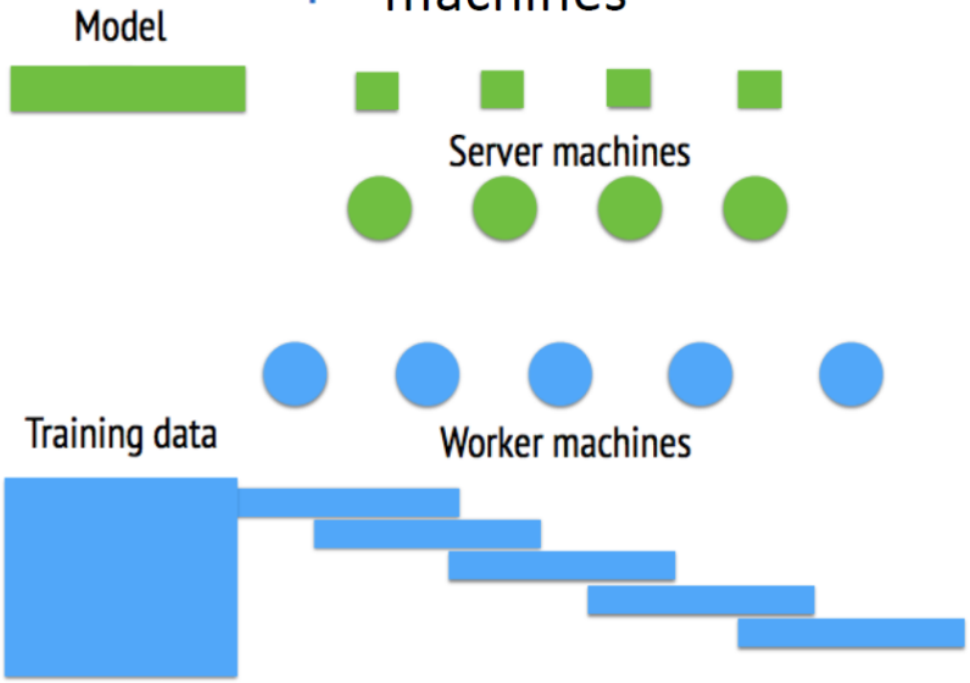
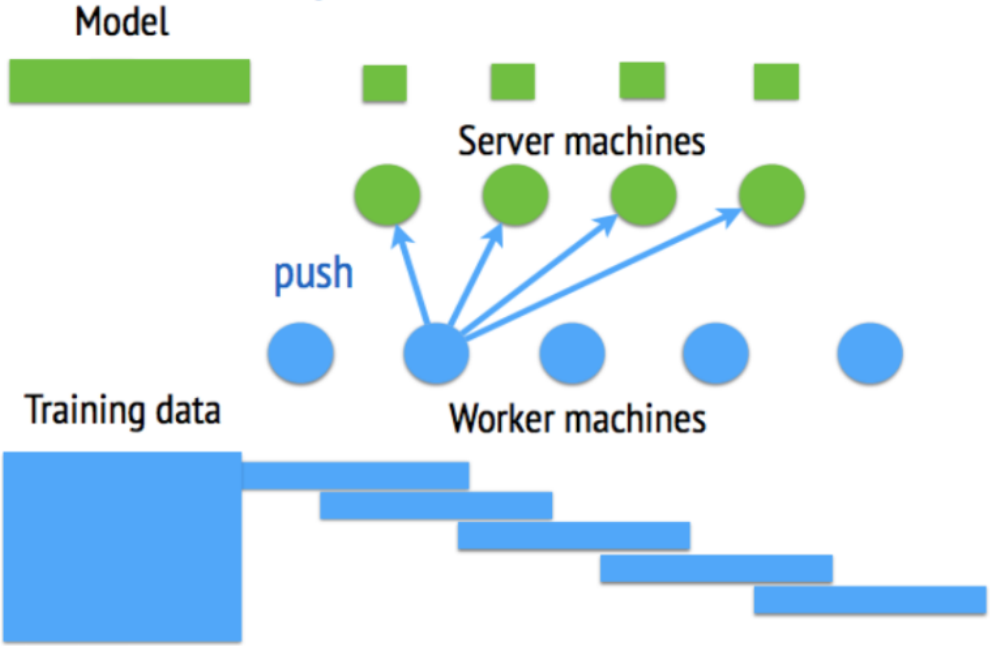


Figure 4: Architecture of a parameter server communicating with several groups of workers.

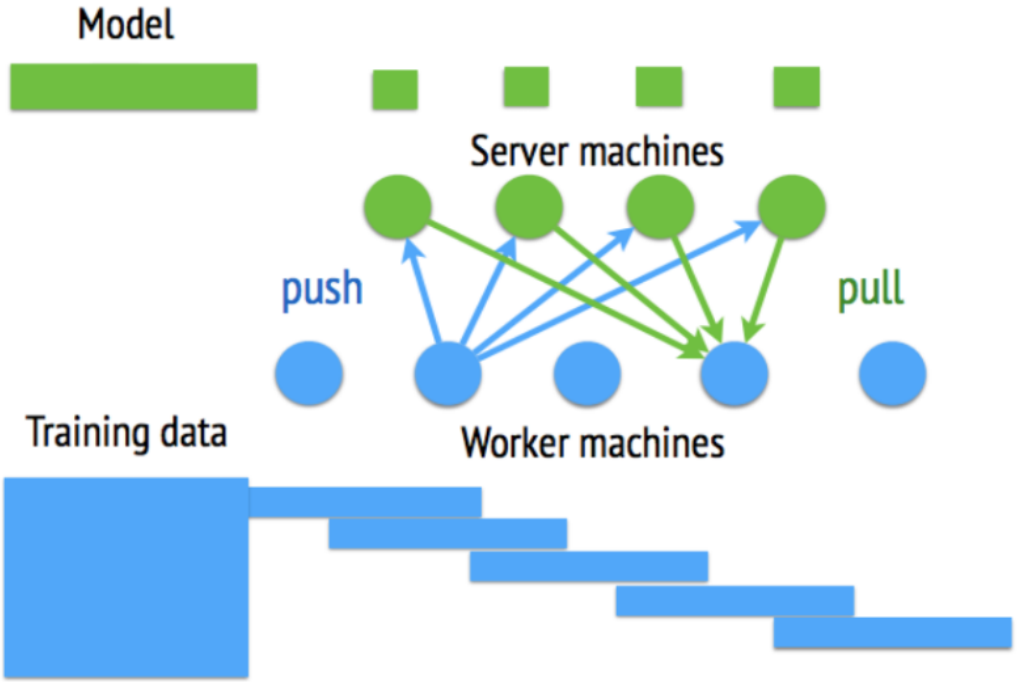
PS partitions model parameters across machines



PS distributes communication and computation for weight updates

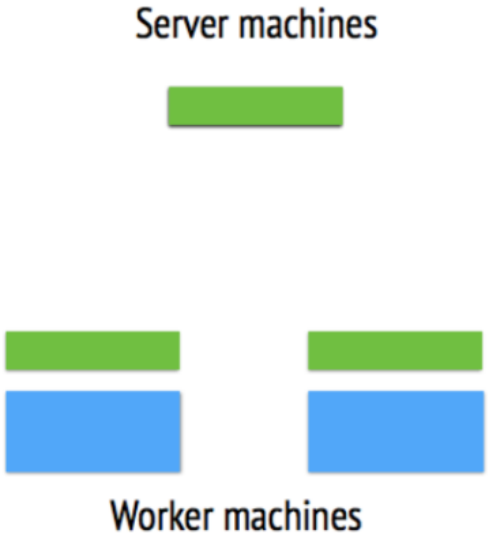


PS distributes communication and computation for weight updates



How data-parallel gradient descent works under PS

Workers **pull** the working set of model



How data-parallel gradient descent works under PS

Workers **pull** the working set of model
Iterate until stop
workers compute gradients

Server machines





Worker machines

How data-parallel gradient descent works under PS

Workers **pull** the working set of model
Iterate until stop
workers compute gradients
workers **push** gradients

Server machines





Worker machines

How data-parallel gradient descent works under PS

Workers **pull** the working set of model
Iterate until stop
workers compute gradients
workers **push** gradients
update model

Server machines





Worker machines

How data-parallel gradient descent works under PS

Server machines



Workers **pull** the working set of model
Iterate until stop
- workers compute gradients
- workers **push** gradients
- update model
- workers **pull** updated model



Worker machines



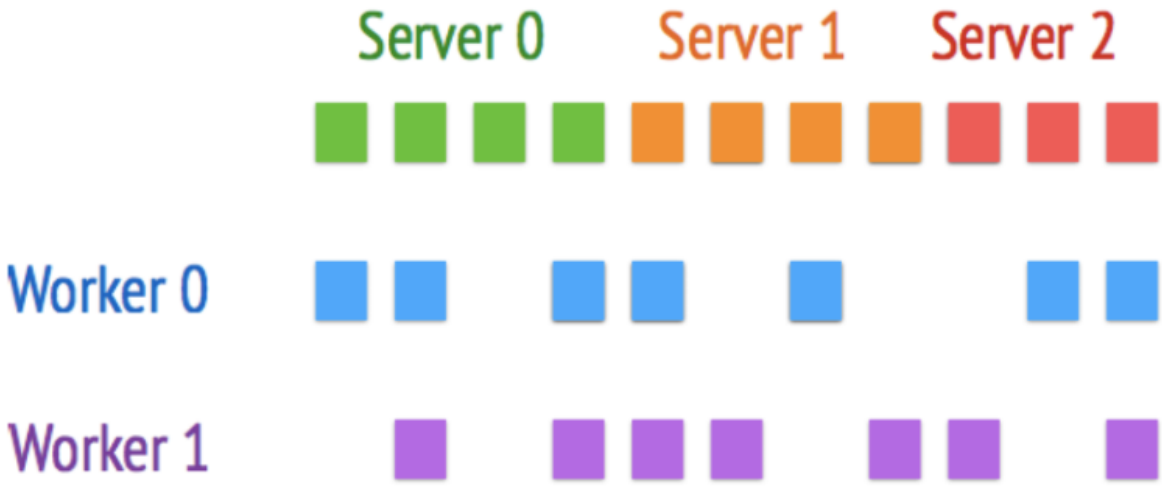




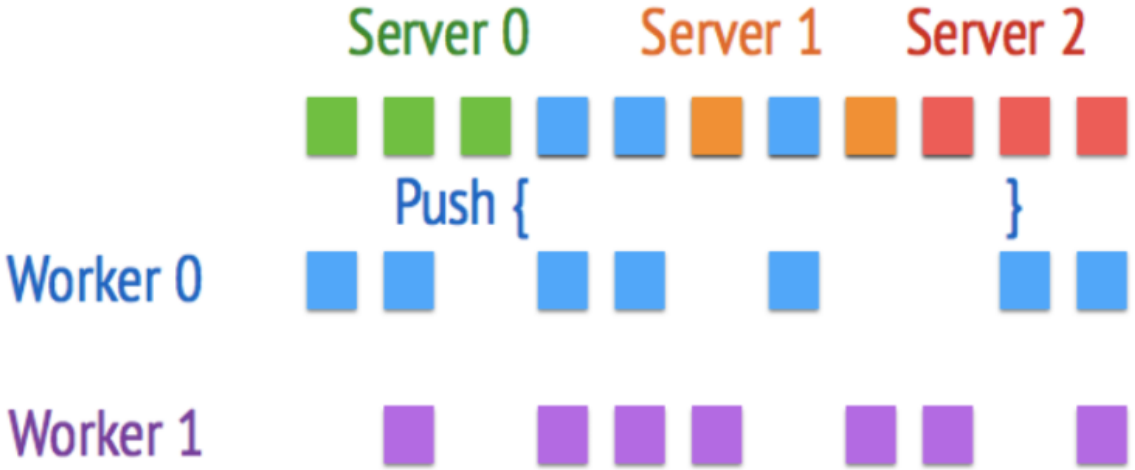
Some additional features provided by PS

- Reduce push/pull communication traffic
  - not all parameters are used/updated by all workers
- Global synchronization may not always be needed

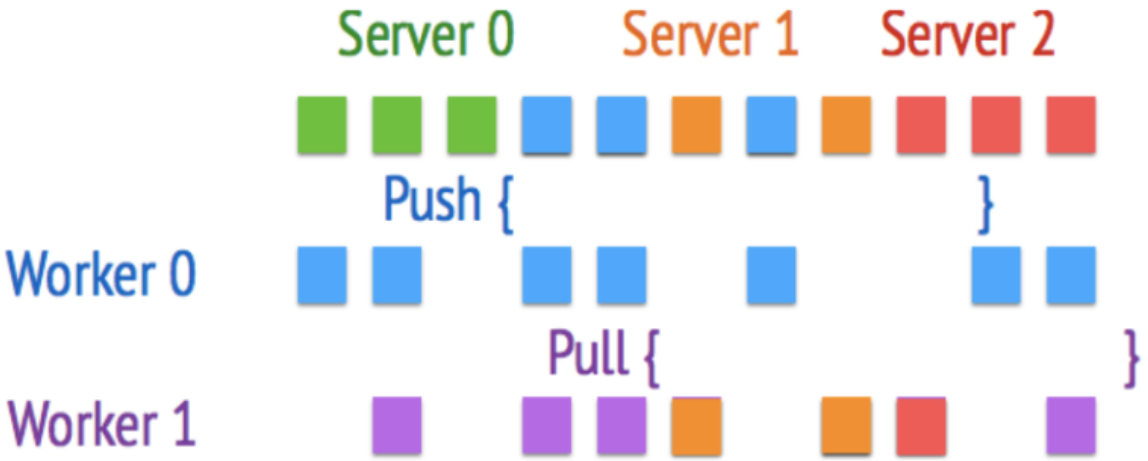
Range-based push and pull



Range-based push and pull



Range-based push and pull



Different consistencies for weights

Sequential / BSP



Bounded delay / SSP
[Langford 09, Cipar 13]



Eventual /
Total asynchronous
[Smola 10]



Consistent Hashing

The parameter server partitions keys much as a conventional distributed hash table does. Keys and server node IDs are both inserted into the hash ring.

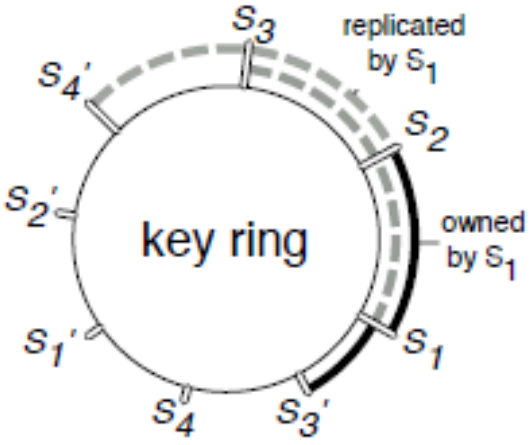


Figure 7: Server node layout.

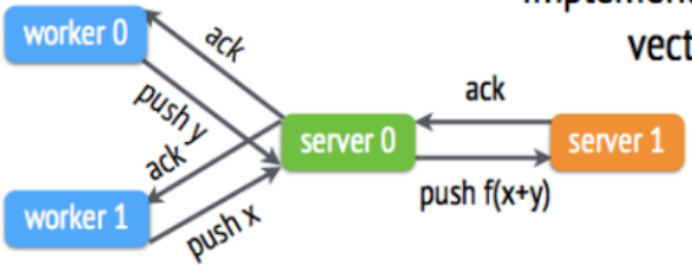
Fault tolerance

- Default replication: Chain replication (consistent, safe)



- Option: Aggregation reduces backup traffic (algo specific)

implemented by efficient vector clock









深度学习



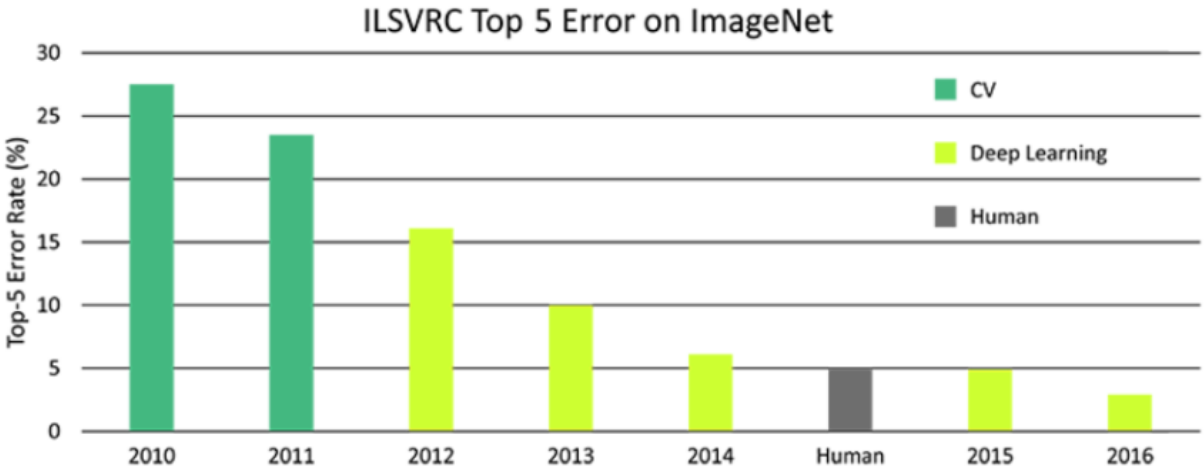




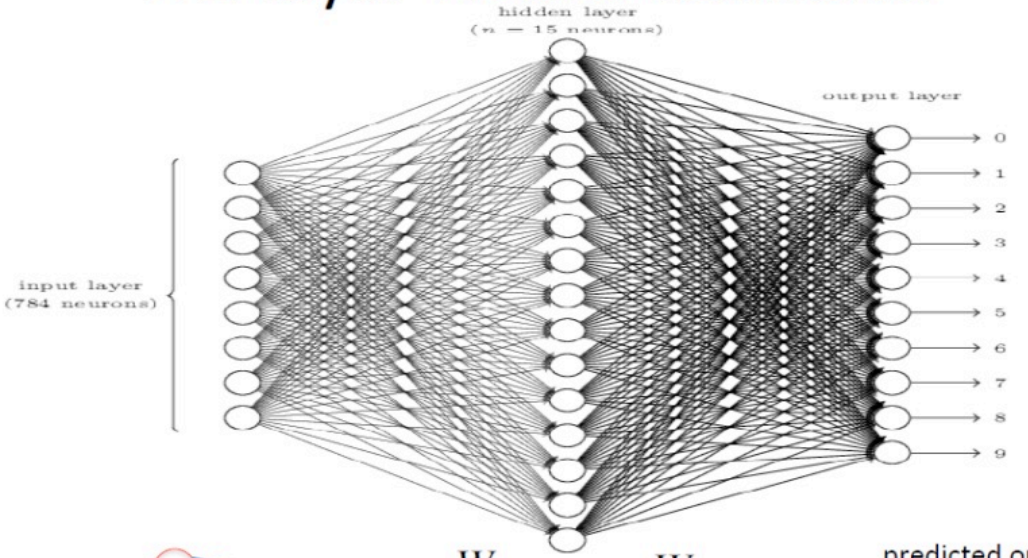
The Deep learning revolution

- Deep neural network has become state-of-art technique for many AI problems:
  - Image recognition
  - Speech recognition
  - Machine translation
  - Question-anwer
  - AlphaGo
  - ...

Why is deep learning so hot/important



A 2-layer Neural networks





x

vector of 784
(flattend 28x28 image)

W_1

784x15 matrix

W_2

15x10 matrix

predicted output y is:

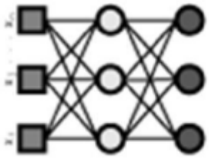
$$h_1 = g(x \cdot W_1)$$

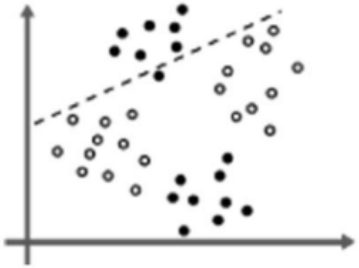
$$h_2 = g(h_1 \cdot W_2)$$

$$y = soft\max(h_2)$$

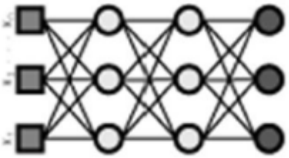
DNNs have many layers

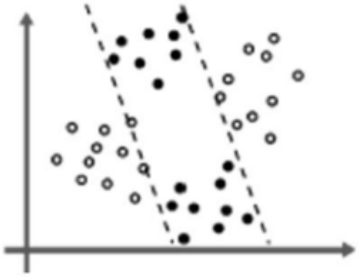
single layer



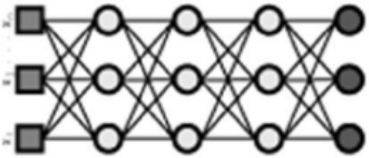


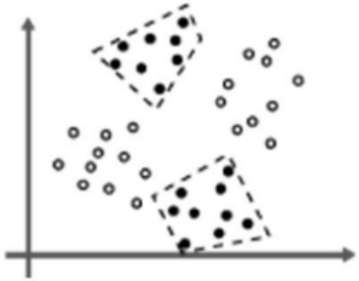
two layer



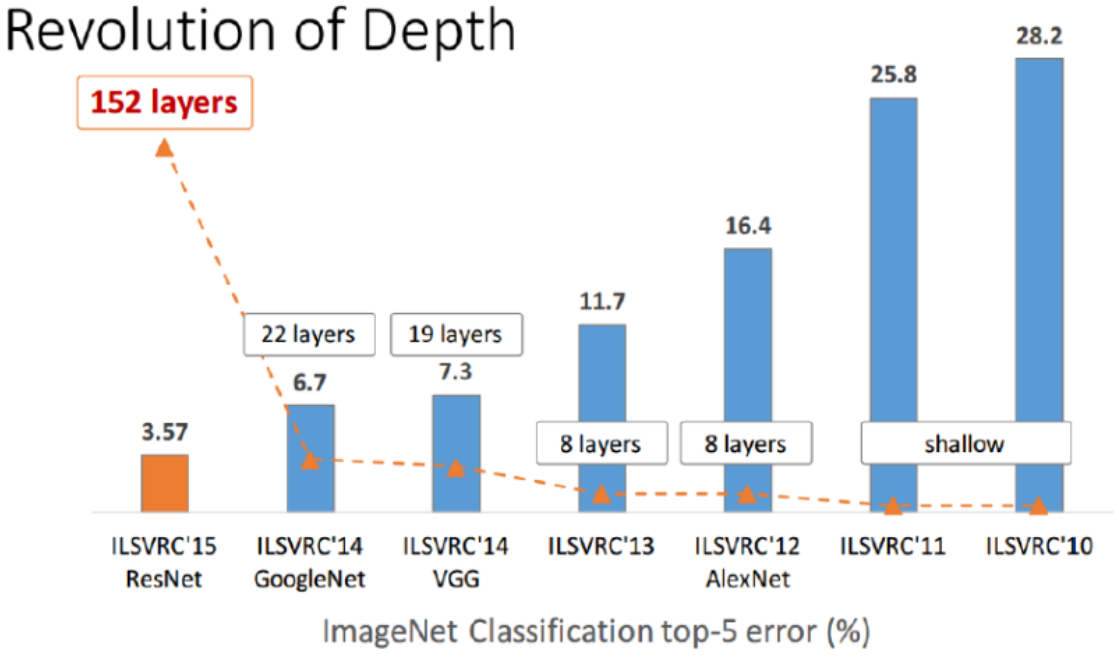


three layer





DNNs have many layers



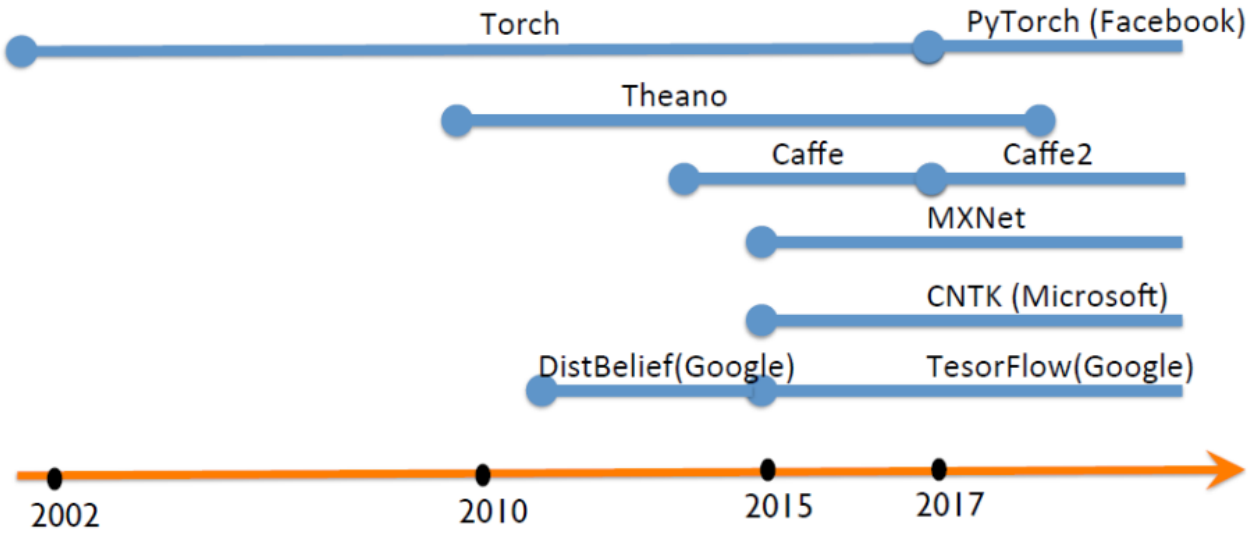




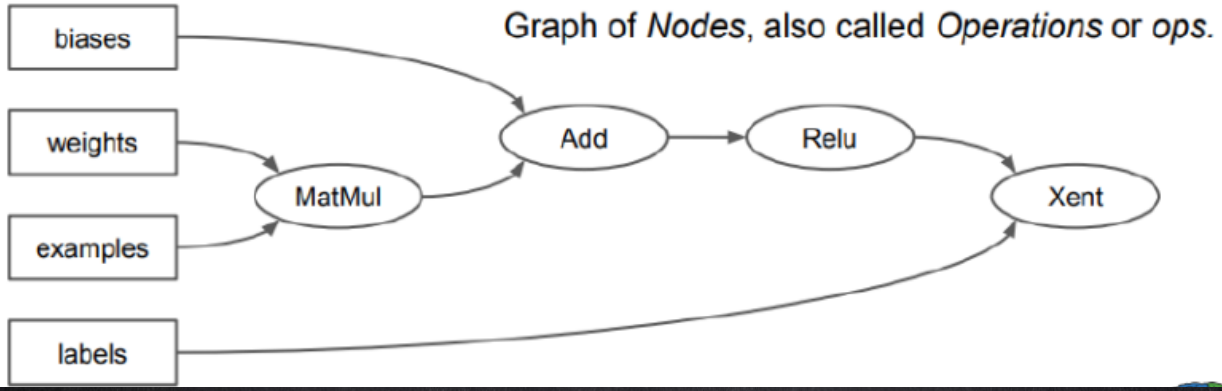
Why deep learning needs its own frameworks

- DNN computation is mostly dense
  - should use tensor (N-dimension array) instead of key-value (Spark) or graph (PowerGraph), to maximize efficiency
- DNN computation is evolving very fast
  - must allow all aspects of computation to be programmable in a high-level language
- DNNs are trained using stochastic gradient descent
  - Calculating gradient by hand is too much, must support auto-differentiation
- GPU acceleration
  - must use fast GPU kernels for convolution, matrix multiplication etc.

Deep learning frameworks

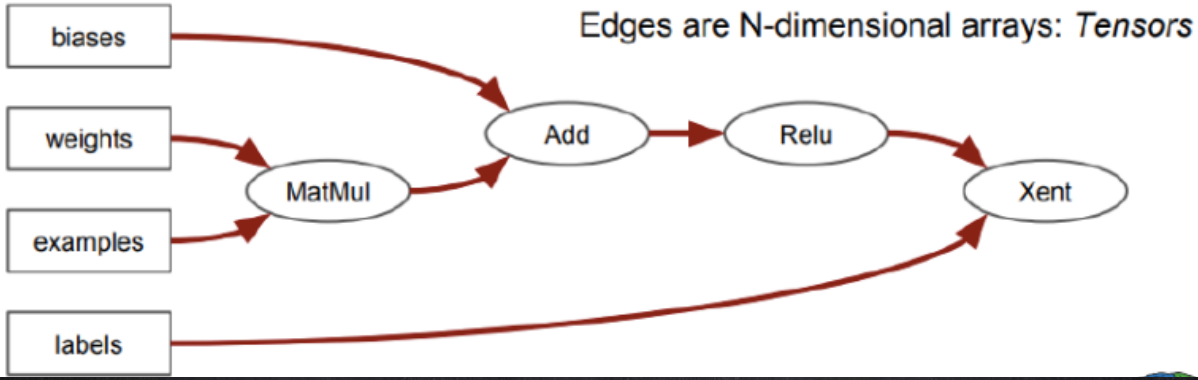


TensorFlow: Computation as a graph



Graph of *Nodes*, also called *Operations* or *ops*.

Computation as a dataflow graph





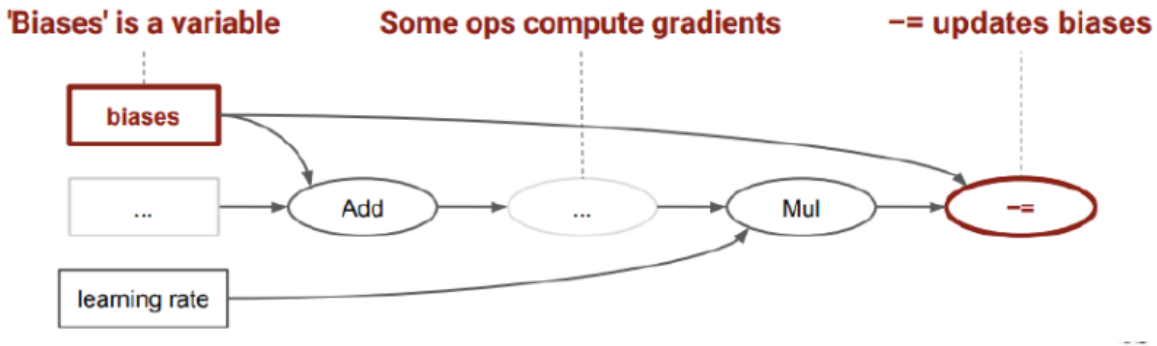


Example TensorFlow program

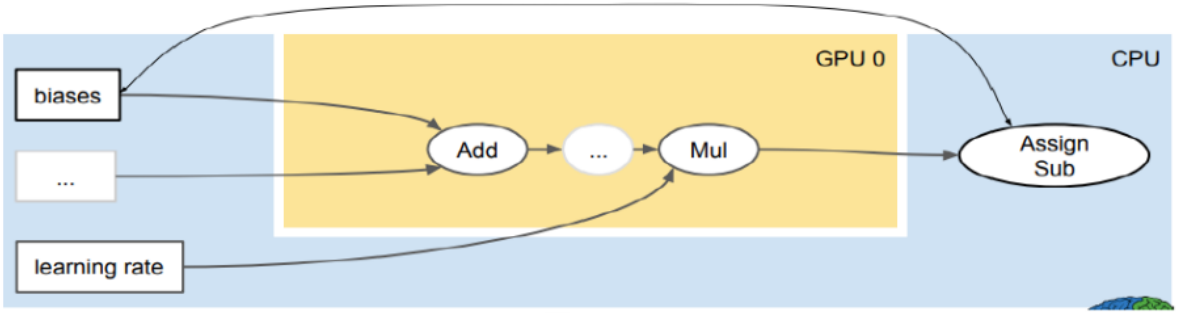
```
import tensorflow as tf
from tensorflow.examples.tutorials.mnist import input_data

mnist = input_data.read_data_sets('MNIST_data', one_hot=True)
x = tf.placeholder("float", shape=[None, 784])
W = tf.Variable(tf.zeros([784,10]))
b = tf.Variable(tf.zeros([10]))
y = tf.nn.softmax(tf.matmul(x, W) + b)
```

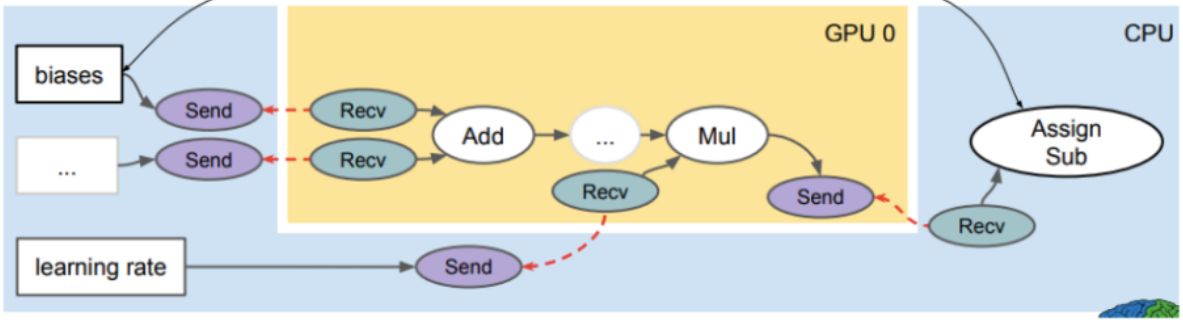
Computation is a dataflow graph with state



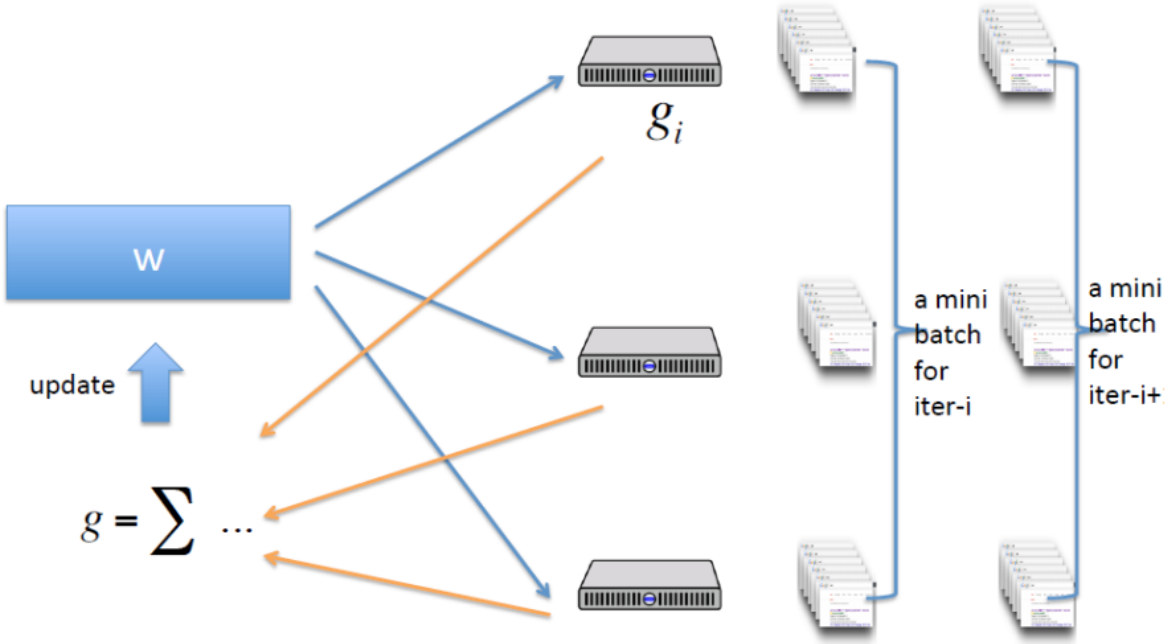
TensorFlow's graph can be distributed



TensorFlow inserts Send/Recv to transfer data across workers/devices



How is DNN training commonly distributed? data parallelism







TensorFlow has no explicit PS

```
with tf.device("/job:ps/task:0"):
  weights_1 = tf.Variable(...)
  biases_1 = tf.Variable(...)

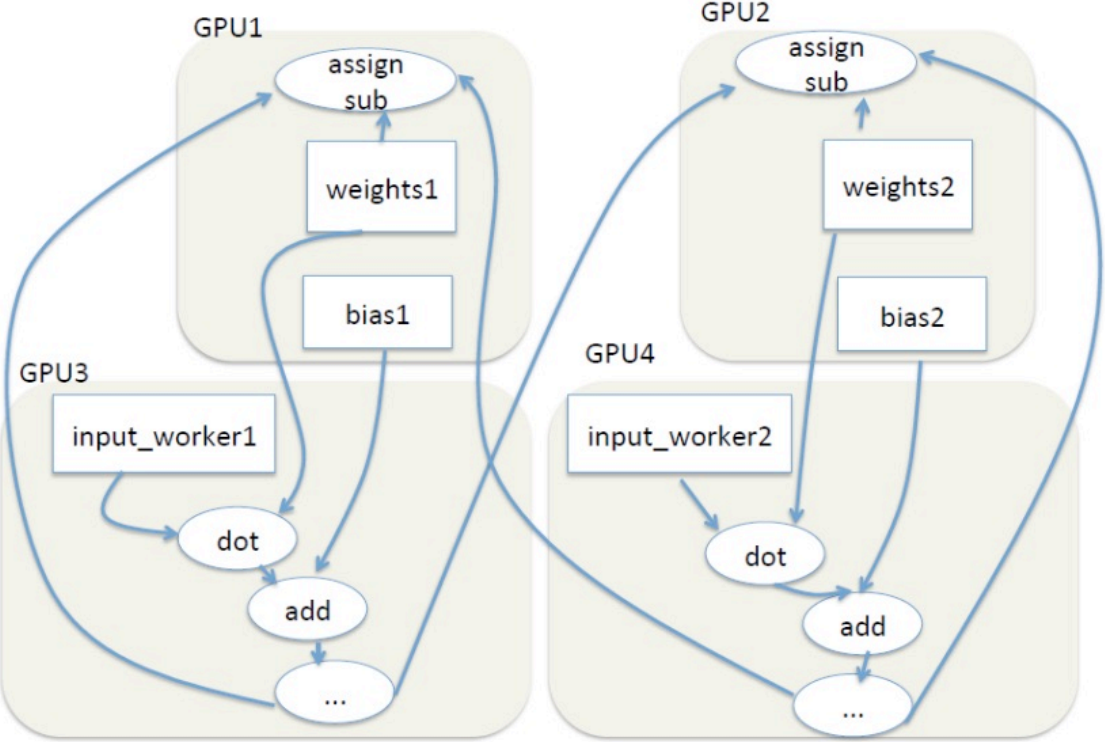
with tf.device("/job:ps/task:1"):
  weights_2 = tf.Variable(...)
  biases_2 = tf.Variable(...)

with tf.device("/job:worker/task:7"):
  input, labels = ...
  layer_1 = tf.nn.relu(tf.matmul(input, weights_1) + biases_1)
  logits = tf.nn.relu(tf.matmul(layer_1, weights_2) + biases_2)
  # ...
  train_op = ...

with tf.Session("grpc://worker7.example.com:2222") as sess:
  for _ in range(10000):
    sess.run(train_op)
```

http://tensorflow.org/deploy/distributed

TensorFlow has no explicit PS







Open questions: What TF did not do so well?

- Data parallelism is not always the right answer
  - Requires larger batch size → negatively affects accuracy and convergence
- TensorFlow's distribution is flexible, but requires a lot of manual efforts
  - Manually partition weights (partition different weight matrices to different devices, partition a single weight matrix to different devices??)
  - How to partition dataflow graph among devices?





Summary: distributed machine learning

- Distributed ML algorithms are diverse
  - some suitable for MapReduce/Spark, GraphLab
  - others require new framework support
- Parameter service is good for ML algorithms optimized with data-parallel gradient descent
- Deep learning frameworks adopt based dataflow graph representation for its efficiency and flexibility
- Distributed training currently rely on data parallelism







分布式深度学习平台

1. floydhub: https://www.floydhub.com/;

2. OpenPAI： https://github.com/Microsoft/pai;

3. PaddlePaddle： http://www.paddlepaddle.org/;

4. PiplineAI： https://pipeline.ai/;

5. Kubeflow： https://www.kubeflow.org/;

6. Polyaxon： https://polyaxon.com/;







谢谢！